

The loop closes: a push becomes ops, a ship becomes a pull request

The win: you can explain, to an interviewer or to yourself, how a webhook that is delivered at most once becomes an event that is worked exactly once, why the diff between an engineer's push and the open document has to be three-way and what happens when it is not, how a job is made idempotent by choosing the right predicate for a unique index, why a commit is recorded before it is pushed, and how a preview that anyone with a link could open was put behind a token without a second image; and you can point at the line of code for every claim.

Verification status, honestly. Every commit in this chapter passed lint, the type checker and the unit tests. The integration suites (webhook, git, push, ship, gitea, sandbox-proxy, preview-proxy) and the three e2e specs (push, conflict, ship) passed on build/epics: on 2026-10-02 the clean-clone gate (make clean-clone at 6700e74, a fresh clone on its own stack running the whole make check) ran 1,200 unit, 387 integration and 44 end-to-end tests, all green. Where this page says "measured", it was measured in a learning test or an earlier epic; where it says "the test asserts", that test ran in the gate. Section 10 walks the same loop by hand.

How to use this page. Every code block is cut from the repo at commit d4eea6f, so what you read is what is committed. It is long: one section a day is a fine pace. Sections 3, 5, 6 and 8 are the ones an architect interview will dig into; section 2 is the one a security review will.

```
./init.sh                                # now also Gitea on 127.0.0.1:3002, worker-git, worker-ship; the seed
pushed, the hook installed
pnpm exec vitest run packages/codegen apps/api/src/webhook.test.ts apps/worker/src/push-ops.test.ts # the pure halves:
parse, the signature, the three-way diff
docker compose logs -f worker-git        # one JSON line per page a push touched: applied, conflict, or
skipped
```

Try it, once the stack is up: section 10 walks the loop by hand, push to canvas and Ship to pull request, with the exact commands and what each should show.

The map: the loop closes

- 1 · Gitea joins the stack, and the sandbox clones through the worker
- 2 · One door with a token: the sandbox proxy, and the public URL
- 3 · Reading the file back: parse is the inverse of generate
- 4 · A public route: the webhook
- 5 · At least once, exactly once, and the second door
- 6 · A push becomes ops: git is a peer
- 7 · A refusal that is visible: the conflict banner
- 8 · Ship: idempotent by index, safe by a push that is never forced
- 9 · Testing a loop that spans four processes and a git server
- 10 · The full loop, by hand
- 11 · What the type system could not see, epic 5 edition

Your turn: three drills

Recall, then test yourself

The map: the loop closes

Epic 4 turned a document into one file and ran it. Epic 5 closes the loop: **an engineer's push to that file becomes ops on the open canvas, a push that breaks the file's fixed shape becomes a banner and nothing else, and one press of Ship becomes one pull request that a second press updates.** Along the way the sandbox lost its network and gained a token, because a preview reachable by anyone holding a document id is not a preview you can put behind a public URL. Nothing in the room changed. Git is one more peer, and like the AI and the preview it writes through the same door every browser tab uses.

STEP	WHERE	AND	WHAT HAPPENS
1	init.sh	scripts/gitea.ts	Gitea in compose; the user <code>@noon</code> , a token kept while it works, the seed as ONE commit from our own objects, a webhook to the api
2	apps/worker/src/sandbox.ts	apps/worker/sandbox/Dockerfile	a sandbox on an <code>--internal</code> network clones from a git bundle; the worker fetched and handed over by <code>docker exec</code>
3	apps/worker/src/sandbox-proxy.ts	apps/web/preview-proxy.ts	the one door to every sandbox: <code>/preview/<document>/<nonce>:<hmac>/</code> , and the same path on the canvas's own origin through tunnel
4	packages/codegen/src/parse.ts		<code>parse(tsx, manifest)</code> : the inverse of <code>generate</code> , or a name reason and a line number
5	apps/api/src/webhook.ts	apps/api/src/app.ts	<code>POST /webhooks/gitea</code> : a public route; HMAC over the raw bytes, then one row
6	packages/db/migrations/0009_git_events.sql	apps/worker/src/git.ts	the inbox with two unique keys; the git peer's mirror, its reconcile a worktree per event
7	apps/worker/src/push-ops.ts	apps/worker/src/push.ts	a three-way diff to minimal ops, submitted through <code>@noon/peer-client</code> as actor <code>git</code>
8	packages/db/migrations/0010_document_conflicts.sql	apps/web/src/ConflictBanner.tsx	a refused push, one row per document, a <code>role="alert"</code> that names the commit and the file
9	packages/db/migrations/0011_ship_jobs.sql	apps/worker/src/ship.ts	Ship: a job whose QUEUED row is unique per document; a commit from git objects alone; a push never forced; one pull request found by <code>head.ref</code>

Four ideas carry the epic. **At least once plus a database key is exactly once** (sections 4 and 5): Gitea does not retry, so there is a second door, and both doors meet at one unique constraint. **Git is a peer, not a writer** (section 6): every change to a document goes through the room, so the git peer submits ops and the room may refuse them one by one. **The diff is three-way** (section 6): what the engineer changed, replayed on the document as it is now, never the pushed file against the document. And **idempotence lives in an index, and the commit is recorded before the push** (section 8). Section 2 is a security boundary that had to be rebuilt before any of this could be shown to anyone.

THIS REPO	JAVA WORLD	PYTHON WORLD
a webhook receiver with a raw-body HMAC and a Postgres unique key as the dedupe	a Kafka consumer with an idempotency key in the same transaction as the write	the same; <code>ON CONFLICT DO NOTHING</code> in psycopg
a reconcile on a timer and on demand, reading a bare mirror	an outbox/inbox pattern with a poller	a Celery beat task that lists remote refs
a three-way tree diff to ops, replayed on the live state	JGit's <code>ThreeWayMerger</code> ; <code>diff3</code>	<code>difflib</code> is two-way; you write the three-way
a commit built from git objects, no checkout, over the CLI	JGit <code>DirCache</code> + <code>CommitBuilder</code>	dulwich; or the same CLI plumbing

1 · Gitea joins the stack, and the sandbox clones through the worker

The stack gains a git server it owns. Gitea runs in compose on the host's loopback (3002: the api and sync have 3000 and 3001), with SQLite, no install page, no registration, sign-in to view, and one permitted webhook host: `api`. Everything `init.sh` does to it is scripted and safe to run again:

make the user `noon` (`admin` is a reserved name, measured in the learning test), keep the worker's token while Gitea still accepts it and mint one otherwise, then bootstrap the repo.

FROM `docker-compose.yml`

```
gitea:
  image: gitea/gitea:1.24
  environment:
    USER_UID: "1000"
    USER_GID: "1000"
    GITEA__security__INSTALL_LOCK: "true" # no install page: init.sh makes the user from the CLI
    GITEA__database__DB_TYPE: sqlite3
    GITEA__server__ROOT_URL: "http://localhost:${GITEA_PORT:-3002}/"
    GITEA__server__DISABLE_SSH: "true" # HTTP only, with the token
    GITEA__service__DISABLE_REGISTRATION: "true"
    GITEA__service__REQUIRE_SIGNIN_VIEW: "true"
    # Gitea delivers webhooks to external hosts only by default: the api, and nothing else, may be dialled.
    GITEA__webhook__ALLOWED_HOST_LIST: api
    # 3002 on the host: 3000 and 3001 are the api's and sync's.
  ports: ["127.0.0.1:${GITEA_PORT:-3002}:3000"]
```

FROM `init.sh`

```
# A secret goes to curl on stdin (-K -), never in its arguments, where any process on this machine can read it.
curl_with() { secret=$1; shift; printf '%s\n' "$secret" | curl -K - "$@"; }
# Gitea's tokens, each in .env under $1: kept while Gitea still answers $4 with it, else made anew as $2 with
# scopes $3 (a new volume forgets every token). Prints the token.
gitea_token() {
  value=$(sed -n "s/^$1=//p" .env)
  if [ "$(curl_with "header = \"Authorization: token ${value:-none}\"" -s -o /dev/null -w '%{http_code}' "$4")" != 200 ]; then
    curl_with "user = \"noon:${GITEA_ADMIN_PASSWORD}\"" -s -o /dev/null -X DELETE "$gitea_url/api/v1/users/noon/tokens/$2"
    value=$(curl_with "user = \"noon:${GITEA_ADMIN_PASSWORD}\"" -fsS -X POST -H 'content-type: application/json' \
      -d '{"name\":\"$2\", \"scopes\":\"$3\"' "$gitea_url/api/v1/users/noon/tokens" | sed -n 's/.*\"sha1\":\"\\([0-9a-f]*\\)\".*\\1/p')
    [ -n "$value" ] || { echo "FAIL: Gitea did not issue the $2 token" >&2; exit 1; }
    { grep -v "^$1=" .env; printf '%s\\n' "$1" "$value"; } > .env.new && mv .env.new .env
  fi
  printf '%s' "$value"
}
# The token that may write: bootstrap creates the repo (write:user) and worker-ship pushes and opens the PR.
# ponytail: worker-ship gets write:user too, which it never uses. Upgrade: a third, write:repository-only token.
GITEA_TOKEN=$(gitea_token GITEA_TOKEN noon-worker '["write:repository","write:user"]' "$gitea_url/api/v1/user")
# The private repo noon/sample-app, the seed pushed into it while it is empty, the push webhook to the api.
```

Read the shell carefully: the password and the token reach `curl` through a config file on stdin (`curl_with`), never as an argument, because arguments are visible to every process on the machine. The bootstrap is a TypeScript script run by Node directly (no build step), and its first job is to make the seed *the same commit everywhere*: `git commit-tree HEAD:seed/sample-app` with a fixed author and date builds a commit from objects already in this repository, so the same tree gives the same id on every machine and every run.

FROM `scripts/gitea.ts`

```
/**
 * The seed as ONE commit whose tree is seed/sample-app at HEAD, made from this repository's own objects:
 * no temp directory, nothing uncommitted. A fixed author and date, so the same tree gives the same commit
 * id on every machine and every run.
 */
export async function seedCommit(): Promise<string> {
  const who = { GIT_AUTHOR_NAME: "noon", GIT_AUTHOR_EMAIL: "noon@localhost", GIT_AUTHOR_DATE: "2026-01-01T00:00:00Z" };
  const env = { ...process.env, ...who, GIT_COMMITTER_NAME: who.GIT_AUTHOR_NAME, GIT_COMMITTER_EMAIL: who.GIT_AUTHOR_EMAIL,
    GIT_COMMITTER_DATE: who.GIT_AUTHOR_DATE };
  return (await exec("git", ["-C", REPO_ROOT, "commit-tree", "-m", "seed: the sample app", "HEAD:seed/sample-app"], { env
  })).stdout.trim();
}

/** Pushes the seed commit as `main` of `remote` (a URL or a local path). The token goes as a header, never into the URL. */
export async function pushSeed(remote: string, auth?: { user: string; token: string }): Promise<string> {
  const commit = await seedCommit();
  const header = auth ? { GIT_CONFIG_COUNT: "1", GIT_CONFIG_KEY_0: "http.extraHeader", GIT_CONFIG_VALUE_0: `Authorization: Basic
${Buffer.from(`${auth.user}:${auth.token}`).toString("base64")}` } : {};
  await exec("git", ["-C", REPO_ROOT, "push", "--quiet", "--", remote, `${commit}:refs/heads/main`], { env: { ...process.env,
    GIT_TERMINAL_PROMPT: "0", ...header } });
  return commit;
}
```

FROM `scripts/gitea.ts`

```
export async function bootstrapGitea(setup: GiteaSetup): Promise<void> {
  const api = async (method: string, path: string, body?: unknown): Promise<Response> => {
```

```

const res = await fetch(`${setup.url}/api/v1${path}`, {
  method,
  headers: { authorization: `token ${setup.token}`, "content-type": "application/json" },
  ...(body === undefined ? {} : { body: JSON.stringify(body) }),
});
// 409 on create is "it exists already", which a second run expects.
if (!res.ok && !(method === "POST" && path === "/user/repos" && res.status === 409)) throw new Error(`Gitea ${method} ${path} ->
${String(res.status)}: ${await res.text()}`);
return res;
};
const repo = `/repos/${setup.user}/${setup.repo}`;
// Private: the worker reads it with the token, and nobody else on this machine reads it at all.
await api("POST", "/user/repos", { name: setup.repo, private: true, default_branch: "main" });
// Only an EMPTY repo gets the seed: after the first run it holds the owner's commits, never overwritten.
const { empty } = (await api("GET", repo)).json() as { empty?: unknown };
if (empty === true) {
  await pushSeed(`${setup.url}/${setup.user}/${setup.repo}.git`, { user: setup.user, token: setup.token });
  // Gitea marks the repo non-empty from a queue after the push returns (measured ~2 s); until then its API
  // answers `branches/main` with a 500 panic. Return only once main is readable, so no caller races it.
  for (const deadline = Date.now() + 30_000; ; await new Promise((wake) => setTimeout(wake, 100))) {
    if (((await api("GET", `${repo}/hooks`)).json() as { empty?: unknown }).empty === false) break;
    if (Date.now() > deadline) throw new Error(`Gitea ${repo}: still empty 30 s after the seed push`);
  }
}
// One hook per URL, its secret kept in step with .env: updated when it exists, made when it does not.
const hook = { active: true, events: ["push"], config: { url: setup.webhook.url, content_type: "json", secret: setup.webhook.secret } };
const hooks = (await api("GET", `${repo}/hooks`)).json() as Hook[];
const ours = hooks.find((each) => each.config.url === setup.webhook.url);
if (ours) await api("PATCH", `${repo}/hooks/${String(ours.id)}`, hook);
else await api("POST", `${repo}/hooks`, { type: "gitea", ...hook });
}

```

Three habits from that file you will use everywhere in this epic. **promisify(execFile)** turns Node's callback-style child process into a promise; **execFile** (not **exec**) takes an argument array, so there is no shell and nothing to quote. **new URL("../", import.meta.url)** is how an ES module finds its own directory (**__dirname** does not exist in modules). And **the token goes to git as an HTTP header through GIT_CONFIG_COUNT / GIT_CONFIG_KEY_0 / GIT_CONFIG_VALUE_0**, never in the URL: a URL lands in **.git/config** of every clone and in every git error message, and the worker's config refuses one that carries credentials.

FROM **apps/worker/src/config.ts**

```

// No credential in it: the URL lands in the mirror's config and in every git error. The token goes as a header.
SEED_REPO: z.string().refine((value) => URL.canParse(value) && ["http:", "https:"].includes(new URL(value).protocol) && new
URL(value).username === "" && new URL(value).password === "", "SEED_REPO must be an http(s) URL without
credentials").default("http://gitea:3000/noon/sample-app.git"),

```

The sandbox cannot reach Gitea, on purpose. Since noon-9gz (section 2) each sandbox sits on its own **--internal** Docker network: no egress, no **host.docker.internal**, no other sandbox. So it cannot clone. The worker, which is on the compose network, keeps a bare mirror per seed URL, fetches it on every start, and hands **main** over as a *git bundle* by **docker exec** into the sandbox's **/tmp**, the way it already hands over the page. The clone's **origin** is that file: no host and no credential anywhere customer code can read.

FROM **apps/worker/src/sandbox.ts**

```

/**
 * Hands the seed to a container that just started: `main` as a git bundle, written into its /tmp, where
 * start.sh waits for it and clones from it. The sandbox has no route to Gitea (its network is --internal,
 * noon-9gz), so the worker, which has one, fetches and hands the history over by `docker exec`, the way it
 * already hands over the page. The clone's `origin` is that file: no host, no credential for customer code
 * to read in .git/config (noon-9gz note e). Written under another name and renamed: start.sh never reads half.
 * Rejected: a route from the proxy to Gitea (customer code would reach every repo the token reads).
 */
async function deliverSeed(docker: string, name: string, seed: SeedRepo, deadline: AbortSignal): Promise<void> {
  const bundle = await bundleOf(seed, deadline);
  await dockerCli(docker, ["exec", "--interactive", name, "sh", "-c", `cat > ${SEED_BUNDLE}.part && mv ${SEED_BUNDLE}.part
${SEED_BUNDLE}`, deadline, bundle]);
}

/**
 * `main` of the seed repo as a bundle, from this process's bare mirror of it, fetched first: every start
 * gets what Gitea has now. One git at a time per mirror. The pid is in the path: test files share a /tmp.
 * ponytail: the mirror lives in /tmp (a restarted worker clones it again) and the bundle is held whole in
 * memory, up to the sandbox's 64m /tmp. Ceiling: repos of that size; upgrade: the git peer's mirror volume
 * (SPEC §2.15) and a bundle piped straight into `docker exec`.
 */
function bundleOf(seed: SeedRepo, deadline: AbortSignal): Promise<Buffer> {
  const mirror = joinPath(tmpdir(), `noon-seed-${String(process.pid)}-${fingerprint(seed.url)}.git`);
  const git = async (...args: string[]): Promise<Buffer> => (await cli("git", args, deadline, { env: gitEnv(seed), name: `git
${args.join(" ")}` })).stdout;
  const next = (mirrors.get(mirror) ?? Promise.resolve(Buffer.alloc(0))).catch(() => undefined).then(async () => {

```

```

    // "--" : a URL is never an option. A clone killed halfway is finished by the next fetch.
    if (existsSync(joinPath(mirror, "HEAD"))) await git("-C", mirror, "fetch", "--quiet", "--prune", "origin");
    else await git("clone", "--quiet", "--mirror", "--", seed.url, mirror);
    return git("-C", mirror, "bundle", "create", "--quiet", "-", "main");
  });
  mirrors.set(mirror, next);
  return next;
}

```

FROM apps/worker/src/sandbox.ts

```

/** git's environment: the token as an HTTP header (GIT_CONFIG_*, git >= 2.31), and never a prompt. */
export function gitEnv(seed: SeedRepo): NodeJS.ProcessEnv {
  const env: NodeJS.ProcessEnv = { ...process.env, GIT_TERMINAL_PROMPT: "0" };
  if (!seed.auth) return env;
  const basic = Buffer.from(`${seed.auth.user}:${seed.auth.token}`).toString("base64");
  return { ...env, GIT_CONFIG_COUNT: "1", GIT_CONFIG_KEY_0: "http.extraHeader", GIT_CONFIG_VALUE_0: `Authorization: Basic ${basic}` };
}

```

FROM apps/worker/sandbox/Dockerfile

```

COPY --chmod=755 <<'EOF' /home/node/start.sh
#!/bin/sh
set -eu
mkdir -p /tmp/vite-temp
if ! git rev-parse --quiet --verify HEAD >/dev/null 2>&1; then
  # The worker's delivery, written whole then renamed. A minute, then give up: the worker has failed.
  waited=0
  while [ ! -f /tmp/seed.bundle ]; do
    waited=$((waited + 1))
    [ "$waited" -le 600 ] || { echo "no seed from the worker in 60 s" >&2; exit 1; }
    sleep 0.1
  done
  rm -rf .git
  git init --quiet --initial-branch=main
  git remote add origin "$SEED_REPO"
  git fetch --quiet origin main
  git checkout --quiet --force -B "$BRANCH" origin/main
fi

```

Two details are the whole reliability story. The bundle is written under another name and renamed (`cat > x.part && mv x.part x`), so `start.sh`, which polls for the file, never reads half of one: a rename is atomic on one filesystem. And `mirrors` is a `Map` from mirror path to *the last fetch-and-bundle promise*: a new start chains onto the previous one with `.catch(() => undefined).then(...)`, so two starts never run `git` in one mirror at once (git takes ref locks and the loser would fail). That is a mutex written as a promise chain, the JavaScript way of "one at a time per key". Section 8 uses the same shape for pull requests.

FROM apps/worker/src/gitea.int.test.ts

```

test("a sandbox clones from Gitea through the worker: Gitea's main, no credential inside, no route to Gitea", async () => {
  const repo = newRepo();
  await bootstrapGitea(setup(repo));
  const id = newDocument();
  await startSandbox(id, options(repo));
  expect(await inSandbox(id, "git", "rev-parse", "origin/main")).toBe(await mainOf(repo));
  expect(await inSandbox(id, "git", "rev-parse", "--abbrev-ref", "HEAD")).toBe(`noon/${id}`);
  // Customer code reads .git/config and its environment: the token is in neither, and origin is a file.
  expect(await inSandbox(id, "git", "remote", "get-url", "origin")).toBe("/tmp/seed.bundle");
  expect(await inSandbox(id, "cat", ".git/config")).not.toContain(TOKEN);
  expect(await docker("container", "inspect", "--format", "{{json .Config.Env}}", sandboxName(id))).not.toContain(TOKEN);
  // And it cannot fetch for itself: neither Gitea by its compose name nor the host's published port.
  for (const url of ["http://gitea:3000/", `http://host.docker.internal:${new URL(GITEA).port}/`]) {
    const probe = `fetch(${JSON.stringify(url)}, { signal: AbortSignal.timeout(3000) })`.then(() => process.exit(0), () => process.exit(7));
    await expect(inSandbox(id, "node", "-e", probe, url)).rejects.toMatchObject({ code: 7 });
  }
}, 90_000);

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>promisify(execFile)</code> : a child process as a promise, argument array, no shell	<code>ProcessBuilder + waitFor()</code> ; JGit if you would rather not shell out	<code>asyncio.create_subprocess_exec</code> ; GitPython
<code>new URL("../", import.meta.url).pathname</code>	<code>getClass().getResource("../")</code>	<code>Path(__file__).resolve().parents[1]</code>
a credential as <code>GIT_CONFIG_*</code> env, never in a URL	JGit's <code>CredentialsProvider</code> ; the same rule about URLs	the same env; never <code>https://user:token@</code>
a promise chain per key as a mutex	<code>ConcurrentHashMap.compute</code> with a lock per key; <code>Semaphore(1)</code>	<code>asyncio.Lock</code> per key
a git bundle over stdin as "the network"	the same file idea; or a shared volume	the same

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A sandbox with no route out could not clone, and routing the proxy to Gitea would have leaked every repo.

The obvious fix after noon-9gz's `--internal` network was a route from the pool's proxy to Gitea. Rejected: customer code runs in the sandbox, and a route to Gitea with the worker's token would let it read every repository that token reads, and would put the proxy on the compose network. A tarball from Gitea's archive endpoint was rejected too: Ship needs history to build on. The bundle keeps the trust boundary where it was: the only thing that enters a sandbox is bytes the worker wrote, and the only thing that leaves is the dev server's answers through the proxy.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The seed image no longer rebuilds on every sample-app edit.

Before E5.1 the sample app was `COPY` 'd into the sandbox image, so any edit to `seed/sample-app` invalidated a six-minute build. Now the image holds Node, baked `node_modules` and our preview entry; the source arrives at start, from Gitea, through the worker. The test suites need no Gitea for this: `sandbox-testing.ts` pushes the same seed commit into a local bare repository, and git fetches a path exactly as it fetches a URL.

2 · One door with a token: the sandbox proxy, and the public URL

This section is two beads (noon-l96, noon-9gz) and a security review, and it had to land before the loop could be shown to anyone outside the machine. The problem: showing the app through a tunnel (ngrok's free plan: *one* hostname, no wildcards) meant the iframe's `http://127.0.0.1:<port>` was the *visitor's* loopback, and mixed content under https besides. So the preview had to ride the canvas's own origin, path-based: `/preview/<document>/...`. And once anyone with the URL can reach a preview, a document id alone must open nothing.

The token. Every sandbox's dev server serves under a base of `/preview/<document>/<nonce>.<hmac>/`, where the nonce is fresh for every container the worker *creates* and the HMAC signs document and nonce under a key only the worker and the pool's proxy hold. Two locks: the proxy refuses any token it did not see signed for that document (a guess, another document's, a bare id), and the sandbox refuses a token of an earlier container of the same document (its base does not match: a bare 404). A URL lives exactly as long as its container.

FROM `apps/worker/src/sandbox-proxy.ts`

```
const PATH = /^\/preview\/([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})\/([0-9a-f]{16})\.([0-9a-f]{32})\/$/u;
/** A sandbox's dev server: its container name (sandbox.ts `sandboxName`), on the network the two share. */
const sandboxAt = (documentId: string): { host: string; port: number } => ({ host: `noon-sandbox-${documentId}`, port: 5173 });

const mac = (key: string, documentId: string, nonce: string): string => createHmac("sha256",
key).update(`${documentId}/${nonce}`).digest("hex").slice(0, 32);

/** The path segment after the document id. `nonce` is 16 hex digits, fresh for every container. */
export const previewToken = (key: string, documentId: string, nonce: string): string => `${nonce}.${mac(key, documentId, nonce)}`;

/** The document a request may reach, or undefined when it may reach nothing. */
export function previewDocument(key: string, url: string): string | undefined {
  const [, documentId, nonce, given] = PATH.exec(url) ?? [];
  if (documentId === undefined || nonce === undefined || given === undefined) return undefined;
  return timingSafeEqual(Buffer.from(mac(key, documentId, nonce)), Buffer.from(given)) ? documentId : undefined;
}
```

Read `previewDocument : timingSafeEqual`, not `===`. A string compare returns at the first differing byte, and the time it takes tells an attacker how many bytes they have right. The regex fixes both strings at 32 hex digits before the compare, because `timingSafeEqual` *throws* on different lengths. The same two lines guard the webhook in section 4.

FROM apps/worker/src/sandbox-proxy.ts

```
// The preview's HMR WebSocket: the same check, then the raw bytes both ways.
server.on("upgrade", (req: IncomingMessage, socket: Duplex, head: Buffer) => {
  socket.on("error", () => { socket.destroy(); });
  const documentId = previewDocument(key, req.url ?? "");
  if (documentId === undefined) {
    socket.end("HTTP/1.1 404 Not Found\r\nContent-Length: 0\r\n\r\n");
    return;
  }
  const to = upstream(documentId);
  const forward = connect(to.port, to.host, () => {
    const headers: string[] = [];
    for (let i = 0; i < req.rawHeaders.length; i += 2) {
      if (req.rawHeaders[i].toLowerCase() !== "host") headers.push(`${req.rawHeaders[i]} ?? ": ${req.rawHeaders[i + 1]} ?? "`);
    }
    forward.write(`${req.method} ?? "GET" ${req.url} ?? "/" HTTP/1.1\r\nHost: ${HOST}\r\n${headers.join("\r\n")}\r\n\r\n`);
    forward.write(head);
    forward.pipe(socket);
    socket.pipe(forward);
  });
  forward.on("error", () => { socket.destroy(); });
  socket.on("close", () => { forward.destroy(); });
});
```

The proxy is Node's standard library and nothing else: `node:http` 's `createServer` for requests, `request` to forward them, and the server's `upgrade` event for WebSockets, where the HTTP handshake is replayed by hand over a raw `node:net` socket and the bytes are piped both ways. It has to be stdlib-only because of *where it runs*: inside the sandbox image, handed to Node as a string (`node --input-type=module-typescript -e :` Node 24 strips the types of a script it was given on the command line too). No second image to build or keep in step; a label on the container names the program and key it runs, so a changed program or key is a container made anew.

FROM apps/worker/src/sandbox.ts

```
const PROXY_PORT = 8080;
const PROXY_PROGRAM = `${readFileSync(new URL("sandbox-proxy.ts", import.meta.url), "utf8")}\nservePreviews({ key: process.env.PREVIEW_KEY ?? "", port: ${String(PROXY_PORT)} })\n`;
```

FROM apps/worker/src/sandbox.ts

```
async function makeProxy(run: Run, pool: string, image: string, key: string, port: number, want: string): Promise<void> {
  const name = proxyName(pool);
  const seen = await run("container", "inspect", "--format", `{{.State.Running}} {{index .Config.Labels "noon.proxy"}}`, name).catch((err: unknown) => {
    if (String(err).includes("No such container")) return undefined;
    throw err;
  });
  if (seen?.trim() === `true ${want}`) return;
  if (seen !== undefined) await run("rm", "--force", name);
  try {
    await run("run", "--detach", "--name", name, "--label", `noon.proxy=${want}`, "--label", `noon.proxy-pool=${pool}`,
      "--restart", "unless-stopped", "--publish", `127.0.0.1:${String(port)}:${String(PROXY_PORT)}`, "--env", `PREVIEW_KEY=${key}`,
      "--read-only", "--cap-drop", "ALL", "--security-opt", "no-new-privileges", "--pids-limit", "64", "--memory", "256m", "--memory-swap", "256m",
      "--entrypoint", "node", image, "--input-type=module-typescript", "-e", PROXY_PROGRAM);
  } catch (err) {
    if (!String(err).includes("is already in use by container")) throw err; // another process made it first
  }
}
```

The key is *derived*, not a new secret: `HMAC(SESSION_TOKEN_SECRET, "noon-sandbox-preview-key")`. The sync server's secret never leaves the worker (the proxy gets only the derived key), and a stack whose `.env` predates the bead starts without a new entry. Key derivation is the general tool here: one root secret, one purpose-named key per use, and no use can be turned into another.

FROM apps/worker/src/config.ts

```
// The preview tokens' key, derived and not a new secret in .env: the sync server's secret never leaves
// this process (the proxy gets only the derived key), and a stack whose .env predates noon-9gz starts.
const previewKey = createHmac("sha256", parsed.SESSION_TOKEN_SECRET).update("noon-sandbox-preview-key").digest("hex");
```

The network. Each sandbox gets a Docker network of its own with `--internal`, and the proxy is joined to every one of them. Two facts were measured before this shape was chosen, and both are in the comment: Docker publishes *no* port for an internal-only container (hence a proxy at all), and turning inter-container traffic off on one shared network cut the proxy off too (hence one network per document). Disk is capped the same way: a read-only root, the working tree a 256 MB tmpfs, `/tmp` 64 MB, both counted against the container's memory, so filling them costs the sandbox its own memory and never the host's disk.

FROM apps/worker/src/sandbox.ts

```
/**
 * The document's own network. --internal: no route out, no host.docker.internal (which reaches the
 * api's x-dev-user, Postgres and Redis on the host's loopback), and no published port either (Docker
 * binds none for an internal-only container, measured), which is why the proxy exists. One per
 * document, not one shared with inter-container traffic off: that would cut the proxy off too (measured).
 * The clone from Gitea comes through the worker (deliverSeed), never by a route to the compose network.
 * ponytail: a network per sandbox spends one of the daemon's address pools each, about 30 by default
 * (some already taken). Ceiling: that many sandboxes at once, daemon-wide; upgrade: 'default-address-pools'
 * with small subnets in the daemon's config, or explicit '--subnet's carved from one range.
 */
async function ensureNetwork(run: Run, name: string, documentId: string, pool: string): Promise<void> {
  await run("network", "create", "--internal", "--label", `${LABEL}=${pool}`, "--label", `noon.document=${documentId}`, name).catch((err:
  unknown) => {
    if (!String(err).includes("already exists")) throw err; // a restart, or another process made it first
  });
}
```

The public URL. The canvas's own Vite dev server forwards `^/preview/`, HTTP and WebSocket alike, to the stack's sandbox proxy on the loopback, path unchanged, with one fixed target. No lookup, no door to any other port, and no check of its own: the sandbox proxy is the check. The api rewrites only the *origin* of the stored URL when `PREVIEW_PUBLIC_URL` is set; the row keeps the loopback truth, and the contract lets the canvas frame exactly two things: this machine's loopback, or its own origin under `/preview/`.

FROM apps/web/preview-proxy.ts

```
/**
 * The running page through the canvas's OWN origin, for when the app is reached through one public URL
 * (a tunnel: noon-196). The canvas frames https://<public host>/preview/<document>/<token>/noon-preview/,
 * and this forwards every /preview/ request, path unchanged, to the stack's sandbox proxy on this
 * machine's loopback (apps/worker/src/sandbox-proxy.ts). That proxy is the check: without the document's
 * token (minted per container, noon-9gz) it is a bare 404, so this rule needs none of its own.
 * HTTP and the preview's own HMR WebSocket alike. One fixed target: no door to any other port here.
 */
export const previewProxy = (sandboxProxy: string): Record<string, ProxyOptions> => ({
  "^/preview/": { target: sandboxProxy, ws: true },
});

/**
 * Vite answers every preflight itself, BEFORE its proxy, by its own origin rule (localhost only). A module
 * request of the preview (origin "null") that is preflighted, because a tunnel adds a header of its own,
 * got a 204 with no Access-Control-Allow-Origin, and a blank frame. Whether "null" may read the preview is
 * the sandbox's decision (its CORS lock): so the preflight goes on, to the proxy. Vite's origin rule is kept,
 * so the canvas's own files stay closed to other sites.
 */
export const previewCors: CorsOptions = { origin: defaultAllowedOrigins, preflightContinue: true };
```

FROM packages/contracts/src/index.ts

```
/**
 * A document's preview as the canvas reads it (F15): its newest sandbox job. 'none' = never opened.
 * 'url' is set only while the job runs and the sandbox answers; null while it (re)starts = "rebuilding".
 * The stored loopback address, or, when the app is reached through one public URL (noon-196), the same
 * path on the canvas's own origin, which carries it as /preview/<document>/<token>/. That host is the
 * api's configuration, not the row's; the canvas frames only its own origin or the loopback.
 */
export const Preview = z.strictObject({
  status: z.enum(["none", ...RunStatus.options]),
  url: PreviewOutput.shape.url
    .refine((url) => onLoopback(url) || (URL.canParse(url) && /^\/preview\/[0-9a-f-]{36}\/[0-9a-f]{16}\.[0-9a-f]{32}\//u.test(new
    URL(url).pathname)), "a preview answers on 127.0.0.1, or under /preview/")
    .nullable(),
});
export type Preview = z.infer<typeof Preview>;
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>timingSafeEqual</code> after a regex fixes the length	<code>MessageDigest.isEqual</code>	<code>hmac.compare_digest</code>
a key derived with HMAC from one root secret	HKDF in JCA; the same idea	<code>hashlib.pbkdf2_hmac</code> / <code>cryptography</code> 's HKDF
<code>server.on("upgrade")</code> and a hand-written 101 handshake over <code>net.connect</code>	Spring Cloud Gateway does it for you; Undertow's proxy handler	aiohttp's <code>web.WebSocketResponse</code> relayed by hand
a proxy program handed to <code>node -e</code> inside another image	a jar copied into the image; no equivalent of "the source is the artifact"	<code>python -c</code> with the module's text: the same trick
Vite's <code>proxy</code> with one target and <code>ws: true</code>	an <code>nginx location</code> with <code>proxy_pass</code> ; the same fixed target rule	the same <code>nginx</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Vite's own 404 named the base, and would have handed the document id to anyone trying ports.

Before the token, the base was `/preview/<document>/<port>/`, and the dev server had to answer *nothing* outside it. Its own 404 page says "did you mean to visit `/preview/<document>/...`": read in Vite's source, measured on a local run. So a plugin with `enforce: "pre"` answers a bare 404 first. With the token the id alone opens nothing anyway, but the plugin stays: a defence you can point at is worth more than one you infer.

FROM `apps/worker/sandbox/Dockerfile`

```
/**
 * The worker's PREVIEW_BASE, /preview/<document>/<token>/: the canvas's dev server can carry this one on its
 * own origin (noon-196). Everything outside it is a bare 404, before Vite's own answer, which NAMES the base
 * ("did you mean to visit /preview/<document>/<token>/...") and would hand this container's token to anyone
 * holding an earlier one of the same document (sandbox-proxy.ts).
 */
const BASE = process.env.PREVIEW_BASE ?? "/";
const onlyUnderTheBase = () => ({
  name: "noon-preview-base",
  enforce: "pre",
  configureServer(server) {
    server.middlewares.use((req, res, next) => {
      if ((req.url ?? "").startsWith(BASE)) return next();
      res.statusCode = 404;
      res.end();
    });
  },
});
```

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Through the tunnel the frame stayed blank: a header of the tunnel's own turned every module request into a preflight, and Vite answered it before its proxy did.

ngrok adds `ngrok-skip-browser-warning`, which makes every module request from the opaque-origin frame a CORS preflight from origin `"null"`. Vite answers every preflight *itself*, before any proxy or plugin, by its default origin rule (localhost only): a 204 with no `Access-Control-Allow-Origin`, and a blank frame. The sandbox, which owns that decision (its CORS lock allows exactly `"null"`), never saw the request. The fix keeps Vite's rule for the canvas's own files and lets the preflight *continue* to the proxy. Rejected: allowing `"null"` on the canvas itself, which would let any sandboxed frame of any site read the canvas's source.

FROM `apps/web/preview-proxy.int.test.ts`

```
// A module request with a header of its own (a tunnel's, like ngrok-skip-browser-warning) is preflied, and
// from origin "null". The canvas's cors refuses that origin: had it answered, the frame would stay blank.
test("a preflight for the preview is the sandbox's to answer, not the canvas's", async () => {
  const path = `/preview/${doc}/${token}@vite/client`;
  const res = await fetch(`${canvas}${path}`, { method: "OPTIONS", headers: { origin: "null", "access-control-request-method": "GET",
"access-control-request-headers": "x-tunnel" } });
  expect(res.headers.get("access-control-allow-origin")).toBe("null");
  expect(seen.at(-1)).toEqual({ url: path, upgrade: false });
});
```

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The 401s on the preview's module requests were the tunnel's basic auth, not ours.

With basic auth on the tunnel, the canvas loaded (the browser had the credentials) and the preview's module requests failed with 401: requests from inside a sandboxed frame carry no credentials. Nothing in this repository was wrong, and the fix was in the tunnel's policy. The lesson is about debugging across boundaries: when a request fails, first establish *which hop* answered. The response's headers name the server; a 401 with the tunnel's realm is not your 401.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The per-org preview cap became exact, and lesson 4's third drill question got a new ending.

Lesson 4 declined to make "at most four previews per org" exact, and named the upgrade: an advisory lock per org. The noon-9gz security review asked for it, and it is a few lines: the `INSERT ... SELECT` runs in a transaction that first takes `pg_advisory_xact_lock` on the org's lock name (`noon:preview:` and the org id), so opens of one org take turns and the count inside the statement is the truth. The ponytail comment named a ceiling and an upgrade path; the review called the upgrade; the shape of the code did not have to change to take it.

FROM `packages/db/src/index.ts`

```
/** `work` in one transaction that holds the advisory lock named `lock`: whatever else takes that lock waits its turn. */
const inTurn = <T>(lock: string, work: (client: PoolClient) => Promise<T>): Promise<T> =>
  inTx(async (client) => {
    await client.query("select pg_advisory_xact_lock(hashtext($1))", [lock]);
    return work(client);
  });
```

FROM `packages/db/src/index.ts`

```
/** One statement, returning ids, holding the org's preview lock: one at a time per org. */
const inOrgTurn = (orgId: string, sql: string, params: unknown[]): Promise<{ id: string }[]> =>
  inTurn(`noon:preview:${orgId}`, async (client) => (await client.query<QueryResultRow>(sql, params)).rows.map((row) => z.object({
    id: z.string() }).parse(row)));
```

What noon-9gz did not do, and says so: the Docker socket the sandbox worker holds stays raw. A socket proxy that filters by API section still allows `create` and `exec`, which is root; an authorization plugin is the real fix, and it is written down as such. A security review that ends with "here is what is still open" is worth more than one that ends with "done".

3 · Reading the file back: parse is the inverse of generate

Keystone 8 has two directions. `generate` (lesson 4) takes a tree to one file of a fixed shape. `parse` takes a file back to a tree, *only while it still has that shape*, and every way out of the shape has a name and a line number, because that is what the conflict banner shows. The file is untrusted input: anyone who can push wrote it.

FROM `packages/codegen/src/parse.ts`

```
/**
 * Keystone 8, the way back: a generated file that an engineer edited and pushed becomes a document
 * again, but ONLY while it still has the shape `generate()` writes. Everything else is a conflict
 * (F16b), and the conflict must say why, so every way out of the shape has its own reason.
 *
 * The file is untrusted input: anyone who can push to the repo wrote it. So it is PARSED, never run
 * (TypeScript's own parser, syntax only, no program and no type checker), sizes are capped before
 * parsing, the walk over the tree is iterative, and a value is accepted only as a literal, read by
 * its syntax, never by evaluating it.
 *
 * The final judge is `generate()` itself: whatever tree came out is handed back to it, so every rule
 * about components, props and tree shape is the SAME rule in both directions. A second copy of those
 * rules here would drift, and a document this accepts but generate refuses would be a push that
 * lands on the canvas and then cannot be generated or shipped.
 *
 * Not decided here: whether an id in the file was removed from the document earlier (id re-use
 * lands an edit meant for the dead node on the new one), and whether the root id matches the open
 * document. Both need the document's history; E5.3b, which turns this tree into ops, has it.
 */
```

Three decisions follow from "untrusted". The file is **parsed, never run**: TypeScript's own parser (`ts.createSourceFile`, syntax only, no program and no type checker), already a dependency of the design-system package, so no new package. Sizes are **capped before parsing** (2 MB, 5000 nodes, depth

64: the room's own defaults, copied not imported, because codegen must not depend on the sync app). And a **value is accepted only as a literal read from its syntax**: a string literal's text, a numeric literal's normalised text, `true`, `false`. Nothing is evaluated.

FROM `packages/codegen/src/parse.ts`

```
type ParseReason =
  | CodegenReason
  | "too_large"
  | "too_deep"
  | "syntax_error"
  | "extra_statement"
  | "hook"
  | "second_export"
  | "not_page_component"
  | "bad_import"
  | "spread"
  | "conditional"
  | "map"
  | "non_literal_prop"
  | "text_child"
  | "expression_child"
  | "not_an_element"
  | "missing_node_id"
  | "bad_node_id"
  | "duplicate_node_id"
  | "duplicate_prop";
export type Parsed = { ok: true; doc: Doc } | { ok: false; reason: ParseReason; detail: string };
```

The reason type is a *union of string literal types*, and it starts with `CodegenReason`: everything `generate` can refuse, `parse` can refuse too, because the last thing `parse` does is hand its tree to `generate`. That is the design's centre: the rules about components, props and tree shape exist *once*, in `generate`, and are the same in both directions. A second copy here would drift, and a document this accepts but `generate` refuses would be a push that lands on the canvas and then cannot be shipped.

FROM `packages/codegen/src/parse.ts`

```
/** A refusal that carries the line it is about. Thrown inside the walk, caught once in `parse`. */
class OutOfShape extends Error {
  readonly reason: ParseReason;
  constructor(reason: ParseReason, message: string) {
    super(message);
    this.reason = reason;
  }
}

/** Never throws, like `generate`: the caller is the git webhook's job handler, and a reason is what the conflict banner shows. */
export function parse(tsx: string, manifest: Manifest): Parsed {
  try {
    if (typeof tsx !== "string") return { ok: false, reason: "malformed_doc", detail: "the file is not text" };
    // Bytes, as git.ts counts the blob: a CJK character is 1 UTF-16 unit and 3 bytes. A UTF-8 string is
    // never shorter in bytes than in units, so the cheap length check first spares encoding a huge file.
    if (tsx.length > MAX_BYTES || new TextEncoder().encode(tsx).length > MAX_BYTES) return { ok: false, reason: "too_large", detail:
    `larger than ${String(MAX_BYTES)} bytes` };
    const doc = read(tsx);
    // The contract is the trust boundary for everything a document may hold: node ids, prop values
    // (control characters, -0), sizes. What the room would refuse from an op is refused here too.
    const contract = Doc.safeParse(doc);
    if (!contract.success) return { ok: false, reason: "malformed_doc", detail: contract.error.issues.map((issue) =>
    issue.message).slice(0, 3).join("; ") };
    const generated = generate(doc, manifest);
    return generated.ok ? { ok: true, doc } : generated;
  } catch (err) {
    if (err instanceof OutOfShape) return { ok: false, reason: err.reason, detail: err.message };
    // TypeScript's parser recurses: nesting far past MAX_DEPTH (still under MAX_BYTES) overflows its stack.
    if (err instanceof RangeError) return { ok: false, reason: "too_deep", detail: "nested too deeply to read" };
    return { ok: false, reason: "malformed_doc", detail: "the file could not be read" };
  }
}
```

`OutOfShape` is a class extending `Error` with a `readonly reason` field assigned in the constructor. Note what it is *not*: a parameter property (`constructor(readonly reason: ...)`), which this repo bans (`erasableSyntaxOnly`), because Node's type stripping cannot erase syntax that generates code. The walk throws it from wherever the shape breaks; `parse` catches once, at the top, and turns it into a value. A `RangeError` there is not a bug: TypeScript's parser recurses, and a file nested far past our depth cap overflows its stack before our cap is checked; that is a reason (`too_deep`), not a crash.

FROM packages/codegen/src/parse.ts

```
// ponytail: 'parseDiagnostics' is internal to TypeScript (stable for a decade, used by most tools).
// The public route is a whole Program just to ask for syntax errors. If it ever disappears the
// guard below refuses every file loudly rather than accepting broken ones.
const diagnostics = (file as unknown as { parseDiagnostics?: unknown }).parseDiagnostics;
if (!Array.isArray(diagnostics)) throw new Error("typescript no longer reports parse diagnostics");
const first = diagnostics[0] as ts.DiagnosticWithLocation | undefined;
if (first) throw new OutOfShape("syntax_error", `line ${String(file.getLineAndCharacterOfPosition(first.start).line + 1)}:
${ts.flattenDiagnosticMessageText(first.messageText, " ")}`);
```

A ponytail with a fence around it: `parseDiagnostics` is internal to TypeScript, stable for a decade and used by most tools, and the public route is a whole `Program` just to ask for syntax errors. The cast goes through `unknown` to a type with an optional field; if the field ever disappears the guard refuses *every* file loudly, rather than accepting broken ones quietly. That is the rule for depending on something you were not promised: make its absence fail closed.

FROM packages/codegen/src/parse.ts

```
// Iterative, like generate: the depth cap is checked as we go, not after a recursion has run out of stack.
const stack: { element: ts.JsxElement | ts.JsxSelfClosingElement; parent: DocNode | null; depth: number }[] = [{ element: jsx, parent: null, depth: 0 }];
while (stack.length > 0) {
  const frame = stack.pop();
  if (!frame) break;
  const { element, parent, depth } = frame;
  if (depth > MAX_DEPTH) throw out(element, "too_deep", `deeper than ${String(MAX_DEPTH)} levels`);
  if (++count > MAX_NODES) throw out(element, "too_large", `more than ${String(MAX_NODES)} elements`);

  const opening = ts.isJsxElement(element) ? element.openingElement : element;
  if (!ts.isIdentifier(opening.tagName) || opening.typeArguments) throw out(element, "not_an_element", "a tag must be a plain component name");
  const tag = opening.tagName.text;
  if (parent === null && tag !== "div") throw out(element, "not_page_component", `the page must return <div ${NODE_ID}="...">, not <${tag}>`);
  if (parent !== null) {
    if (tag === ROOT_COMPONENT) throw out(element, "reserved_component", `${ROOT_COMPONENT} is the page component's own name`);
    if (!/^([A-Z])/u.test(tag)) throw out(element, "unknown_component", `<${tag}> is not a design-system component`);
    if (!imported.has(tag)) throw out(element, "bad_import", `<${tag}> is used but not imported from ${DESIGN_SYSTEM}`);
  }

  const { id, props } = readAttributes(opening, out);
  if (Object.hasOwn(nodes, id)) throw out(element, "duplicate_node_id", `${NODE_ID}="${id}" appears twice`);
  const node: DocNode = { id, component: parent === null ? ROOT_COMPONENT : tag, props, parentId: parent?.id ?? null, children: [] };
  nodes[id] = node;
  if (parent) parent.children.push(id);
  else rootId = id;

  const children = ts.isJsxElement(element) ? childElements(element, out) : [];
  // Reversed, because a stack hands back what went on last: children are visited, and listed, in order.
  for (const child of children.reverse()) stack.push({ element: child, parent: node, depth: depth + 1 });
}
```

FROM packages/codegen/src/parse.ts

```
/** A prop's value, read from its syntax only: a string, a finite number (optionally negated), true or false. */
function literal(name: string, attribute: ts.JsxAttribute, out: Out): PropValue {
  const init = attribute.initializer;
  if (!init) return true; // '<Button disabled />' is JSX for disabled={true}
  if (ts.isStringLiteral(init)) {
    // A quoted JSX attribute has no escapes but DOES decode HTML entities when compiled: the text
    // TypeScript hands back is the raw one, so a '&' would mean one thing here and another in the page.
    if (init.text.includes("&")) throw out(init, "non_literal_prop", "a quoted attribute with & is decoded as HTML; write it as {`...`}");
    return init.text;
  }
  const expression = ts.isJsxExpression(init) ? init.expression : undefined;
  if (expression) {
    if (ts.isStringLiteral(expression)) return expression.text;
    if (expression.kind === ts.SyntaxKind.TrueKeyword) return true;
    if (expression.kind === ts.SyntaxKind.FalseKeyword) return false;
    const negative = ts.isPrefixUnaryExpression(expression) && expression.operator === ts.SyntaxKind.MinusToken;
    const operand = negative ? expression.operand : expression;
    if (ts.isNumericLiteral(operand)) {
      // TypeScript normalises the literal's text (0x10 -> "16", separators dropped), so Number() reads a decimal.
      const value = Number(operand.text);
      if (Number.isFinite(value)) return negative ? -value : value;
    }
  }
}
throw out(init, expression ? reasonFor(expression, "non_literal_prop") : "non_literal_prop", `${name} must be a literal string, number
```

```
or boolean`);
}
```

FROM `packages/codegen/src/parse.test.ts`

```
test("unit:parse-roundtrip: formatting is not shape (quotes, line breaks, comments, a self-closed empty element)", () => {
  const doc = build(add("s", ROOT_ID, "Stack", { gap: 16 }), add("b", "s", "Button", { label: "Pay", disabled: true }));
  const reformatted = `// an engineer's comment
import {Stack, Button} from '../design-system/index.ts'

/** the page */
export function Page() {
  return <div data-node-id='root'>
    {/* a comment child */}
    <Stack
      gap={0x10}
      data-node-id="s"
    >
      <Button label="Pay" disabled data-node-id="b"></Button>
    </Stack>
  </div>
}
`;
  expect(parsed(reformatted)).toEqual(doc);
});
```

FROM `packages/codegen/src/parse.test.ts`

```
test("unit:parse-shape-breakers: nesting deep enough to exhaust the parser's stack is a reason, not a crash", () => {
  const levels = 250_000; // 7 bytes a level: under the size cap, so it is the parser that meets it
  expect(parse(page(`${"<a>".repeat(levels)}${"</a>".repeat(levels)}`), manifest)).toMatchObject({ ok: false, reason: "too_deep" });
});
```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

`ts.createSourceFile(..., ScriptKind.TSX)` : a syntax tree, no checker

JavaParser / ANTLR: a parse tree, no symbol resolution

`ast.parse`

a value read from a literal's syntax

`LiteralExpr.getValue()`

`ast.literal_eval` : the same promise, evaluate nothing

`class OutOfShape extends Error { readonly reason }` , assigned in the body

a checked exception with a field

an exception class with an attribute

as unknown as `{ parseDiagnostics?: unknown }` : a cast through unknown , then a runtime check

reflection on a private field, then `instanceof`

`getattr(node, "_private", None)` , then a type check

`ts.forEachChild(node, function find(n) { ... return ts.forEachChild(n, find); })` : a named function expression to recurse

a recursive Visitor

`ast.walk` or a recursive helper

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A quoted attribute containing `&` means one thing in the parser and another in the page.

JSX string attributes have no backslash escapes but *do* decode HTML entities when compiled: `label="a & b"` renders as `a & b` , while TypeScript's parser hands back the raw text. A round trip through such a value would silently change the document. So a quoted attribute with `&` is refused with the fix in the message (write it as `{ "... }`), and `generate` only ever writes expression attributes. Lesson 4 had the mirror-image bug: `-0` written as `0` .

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A byte-exact comparison against a fresh generation was rejected as the shape test.

The tempting definition of "in shape" is "equals what `generate` writes". It would call every reformat, every reordered import and every comment a conflict, and engineers reformat. Shape is structural: one import, one exported component, a tree of elements with literal props. The tests say it in one line: *formatting is not shape*. What byte-equality is right for is Ship (section 8), where the file is *ours*.

Principal-level point: when a system has an encoder and a decoder, make one of them the judge. Here the decoder ends by calling the encoder, so "parses" implies "generates", and there is exactly one place a rule about pages lives. The alternative, two implementations kept in step by tests, is the design most codebases have and most codebases regret.

4 • A public route: the webhook

`POST /webhooks/gitea` is the first route in this api with no user behind it. Gitea has nobody to send. So the route assumes a stranger wrote the request until the signature says otherwise, and reads *nothing* from the body before that.

FROM `apps/api/src/webhook.ts`

```
const HEX_SIGNATURE = /^[0-9a-f]{64}$/;

/**
 * True when `header` is the hex HMAC-SHA256 of these exact bytes under `secret` (X-Gitea-Signature,
 * learning-tests/gitea FINDINGS 3). The RAW body is signed: a parse-and-reserialize would verify
 * something Gitea never sent. Compared in constant time; a malformed header is refused before any compare.
 */
export function signatureMatches(raw: Buffer, header: string | undefined, secret: string): boolean {
  if (header === undefined || !HEX_SIGNATURE.test(header)) return false;
  const expected = createHmac("sha256", secret).update(raw).digest();
  return timingSafeEqual(expected, Buffer.from(header, "hex")); // both 32 bytes: the regex fixed the length
}
```

The *raw* bytes are signed. A parse-and-reserialize (`JSON.stringify(JSON.parse(body))`) would verify something Gitea never sent: one byte of whitespace is another body. The regex first, then `timingSafeEqual` : same two lines as the proxy's token, same reasons. And note the argument order: `header: string | undefined` , because a missing header is a normal case, not an exception.

FROM `apps/api/src/app.ts`

```
// Both limits refuse by Content-Length before a byte is read, and count the bytes of a body sent without one.
const smallBodies = bodyLimit({ maxSize: MAX_BODY_BYTES, onError: (c) => fail(c, 413, "payload_too_large") });
const webhookBodies = bodyLimit({ maxSize: MAX_WEBHOOK_BYTES, onError: (c) => fail(c, 413, "payload_too_large") });
app.use(async (c, next) => {
  const webhook = c.req.path === WEBHOOK_PATH;
  // A declared length over the cap is refused here, without touching the body: bodyLimit opens it as a stream
  // first, which stalls @hono/node-server's drain of the unread bytes, and the server then cuts a socket the
  // client has already pooled for its next request (ECONNRESET).
  if (Number(c.req.header("content-length")) > (webhook ? MAX_WEBHOOK_BYTES : MAX_BODY_BYTES)) return fail(c, 413, "payload_too_large");
  return (webhook ? webhookBodies : smallBodies)(c, next);
});
```

FROM `apps/api/src/app.ts`

```
// E5.3a: Gitea's push webhook, public (Gitea has no user to send). The HMAC over the RAW bytes is the only
// authentication, and nothing in the body is read before it passes. One push is one row: the delivery id
// and (branch, commit) are both unique keys in Postgres, so a redelivery, a replay and a reconcile that got
// there first all end as 'duplicate'. The git peer does the work; this answers Gitea within its 5 s.
app.post(WEBHOOK_PATH, async (c) => {
  if (webhookSecret === undefined) return notFound(c);
  const raw = Buffer.from(await c.req.arrayBuffer());
  if (!signatureMatches(raw, c.req.header("x-gitea-signature"), webhookSecret)) return fail(c, 401, "unauthenticated");
  // Not covered by the signature: a replayed body under a made-up id still meets the (branch, commit) key.
  const delivery = c.req.header("x-gitea-delivery");
  if (delivery === undefined || !/^[x21-x7e]{1,100}$/i.test(delivery)) return fail(c, 400, "invalid_body", [{ field: "x-gitea-delivery", message: "required: 1 to 100 printable characters" }]);
  if (c.req.header("x-gitea-event") !== "push") return c.json({ result: "ignored", reason: "not_a_push" });
  const push = readPush(raw);
  if (push.kind === "invalid") return fail(c, 400, push.error);
  if (push.kind === "ignored") return c.json({ result: "ignored", reason: push.reason });
  const recorded = await db.gitStore().record({ ref: push.ref, before: push.before, after: push.after, deliveryId: delivery });
  return recorded ? c.json({ result: "recorded" }, 202) : c.json({ result: "duplicate" });
});
```

Order of the checks, and why each is where it is. **Body size before anything is read:** a push payload lists commits, and a big push is over the api's 64 KB limit, so this route alone gets 1 MiB; the middleware picks the limit by path. **No secret configured: 404**, the route does not exist, and the reconcile alone notices pushes. **Signature**, from the raw `arrayBuffer` . **Delivery id:** required, printable, at most 100 characters, because it becomes a unique key. **Event name:** anything but `push` is ignored. Then, and only then, the body is parsed.

FROM `apps/api/src/webhook.ts`

```
const Sha = z.string().regex(/^[0-9a-f]{40}|[0-9a-f]{64}$/); // sha1 or sha256 object ids, lowercase as git prints them
const ZERO = /^0+$/;
/**
```

```

* A branch as git would name it, narrowed to what a person pushes: it becomes a row, a log line and, in
* E5.3b, a git argument. Anything else (tags, notes, odd names) is ignored, not stored.
*/
const BRANCH = /^refs\/heads\/(?:!-)(?!.*\.\.)(?!.*\/\w)[A-Za-z0-9._/-]{1,200}$/;
const Push = z.object({ ref: z.string(), before: Sha, after: Sha });

```

FROM apps/api/src/webhook.ts

```

/** What a VERIFIED push body says. Only ref/before/after are read: the commit list is truncated for big pushes, so the git peer reads the
mirror. */
export function readPush(raw: Buffer): PushReading {
  let json: unknown;
  try {
    json = JSON.parse(raw.toString("utf8"));
  } catch {
    return { kind: "invalid", error: "invalid_json" };
  }
  const parsed = Push.safeParse(json);
  if (!parsed.success) return { kind: "invalid", error: "invalid_body" };
  const { ref, before, after } = parsed.data;
  if (!BRANCH.test(ref)) return { kind: "ignored", reason: "not_a_branch" };
  if (ZERO.test(after)) return { kind: "ignored", reason: "deleted" };
  // Registering the hook fires a push whose `before` is all zeros (FINDINGS 2). A real new branch looks the
  // same and is ignored too: the git peer's reconcile finds it in the mirror (SPEC §2a).
  if (ZERO.test(before)) return { kind: "ignored", reason: "synthetic" };
  return { kind: "event", ref, before, after };
}

```

Only `ref`, `before` and `after` are read. Gitea truncates the commit list for big pushes, so the file lists in the payload are not trusted for anything; the git peer reads its mirror instead. Two pushes are *ignored* rather than stored: a deleted branch (`after` all zeros), and the push Gitea fires the moment a hook is registered (`before` all zeros, learning-tests/gitea FINDINGS 2). A real new branch looks exactly like the registration push, so it is ignored too, and the reconcile finds it. The `BRANCH` regex with its three negative lookaheads (`(?!-)` , `(?!.*\.\.)` , `(?!.*\/\w)`) narrows a ref name to what a person pushes, because the name becomes a row, a log line and, in section 6, a git argument.

FROM apps/api/src/webhook.test.ts

```

test("anything that is not exactly 64 lowercase hex digits is refused, never compared", () => {
  const body = push();
  const good = sign(body);
  for (const bad of [undefined, "", good.slice(0, 63), `${good}0`, good.toUpperCase(), `sha256=${good}`, ` ${good}`, "z".repeat(64),
`${good.slice(0, 62)}é`]) {
    expect(signatureMatches(body, bad, secret), JSON.stringify(bad)).toBe(false);
  }
});

```

FROM apps/api/src/webhook.int.test.ts

```

test("the same delivery again, and the same push replayed under a new delivery id, record nothing more", async () => {
  const body = pushBody();
  expect(await deliver(body, { "x-gitea-delivery": "same" })).status().toBe(202);
  const again = await deliver(body, { "x-gitea-delivery": "same" });
  expect(again.status).toBe(200);
  expect(await again.json()).toEqual({ result: "duplicate" });
  // The delivery id is not signed: a captured body replayed under a fresh id meets the {branch, commit} key.
  expect(await (await deliver(body, { "x-gitea-delivery": "fresh" })).json()).toEqual({ result: "duplicate" });
  // Ten at once: Postgres's unique key decides, not a look-then-insert in the api.
  const raced = await Promise.all(Array.from({ length: 10 }, () => deliver(pushBody({ before: B, after: A })))));
  expect(raced.map(r => r.status).sort()).toEqual([200, 200, 200, 200, 200, 200, 200, 200, 200, 202]);
  expect(await events()).toHaveLength(2);
});

```

FROM apps/api/src/webhook.int.test.ts

```

test("a body over 1 MiB is refused by its length before it is read; a big but real push is taken", async () => {
  const big = pushBody({ commits: Array.from({ length: 400 }, (_, i) => ({ id: B, message: "m".repeat(400), modified:
[`${src}/${String(i)}.ts`] }))) });
  expect(big.length).toBeGreaterThan(64 * 1024); // more than every other route accepts
  expect(await deliver(big)).status().toBe(202);
  const huge = pushBody({ padding: "x".repeat(1024 * 1024) });
  const res = await deliver(huge);
  expect(res.status).toBe(413);
  expect(await events()).toHaveLength(1);
});

/** A request on `agent`; resolves with its status and the socket that carried it. */
function send(agent: Agent, path: string, init: { method: string; body?: string; headers?: Record<string, string> }): Promise<{ status:
number; socket: unknown }> {

```

```

return new Promise((resolve, reject) => {
  const req = request(`${ctx.server.url}${path}`, { agent, method: init.method, headers: { "x-dev-user": "tester@example.com",
  ...init.headers } }, (res) => {
    const socket = res.socket;
    res.resume().on("end", () => { resolve({ status: res.statusCode ?? 0, socket }); }).on("error", reject);
  });
  req.on("error", reject).end(init.body);
});
}

// The gate's ECONNRESET: the limit opened the body as a stream before refusing it, so the server could not drain what
// it had not read and cut the socket 500 ms later (@hono/node-server's drain timeout), after the client had pooled it.
test("a body refused by its length leaves its connection usable: the next request on it is answered", async () => {
  const huge = pushBody({ padding: "x".repeat(1024 * 1024) });
  const refusals = [
    { path: "/webhooks/gitea", headers: { "content-type": "application/json", "x-gitea-event": "push", "x-gitea-delivery": randomUUID(),
    "x-gitea-signature": sign(huge) }, body: huge },
    { path: "/orgs", headers: { "content-type": "application/json" }, body: JSON.stringify({ name: "x".repeat(70 * 1024) }) },
  ];
  for (const { path, headers, body } of refusals) {
    const agent = new Agent({ keepAlive: true, maxSockets: 1 });
    try {
      const refused = await send(agent, path, { method: "POST", headers, body });
      expect(refused.status, path).toBe(413);
      const next = await send(agent, "/orgs", { method: "POST", headers: { "content-type": "application/json" }, body: JSON.stringify({
name: "After" }) });
      expect(next.status, path).toBe(201);
      expect(next.socket === refused.socket, `${path}: the same connection`).toBe(true);
    } finally {
      agent.destroy();
    }
  }
});

```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

`Buffer.from(await
c.req.arrayBuffer())`: the bytes as sent

`@RequestBody byte[]`, or a
`ContentCachingRequestWrapper`

`await request.body()` in FastAPI

`bodyLimit` chosen per path by a
middleware

`spring.servlet.multipart.max-request-size` is
global; a filter per path

Starlette middleware; nginx
`client_max_body_size` per location

`PushReading`: a discriminated union with
`kind`

a sealed interface with three records

a `Union` of dataclasses, matched with
`match`

`string | undefined` for a header that may
be absent

`Optional<String>`

`str | None`

202 recorded, 200 duplicate, 401
unauthenticated, 413 too large

the same codes

the same codes

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The delivery id is not signed, so a captured body replayed under a fresh id would have been a new event.

`X-Gitea-Delivery` is a header, and the HMAC covers only the body. Anyone who captured one delivery could replay the body with a made-up id and pass every check above. Dedupe on the delivery id alone is therefore not dedupe. The second unique key, `(ref, after_sha)`, is what makes the replay a `duplicate`, and it is also where the webhook and the reconcile meet (section 5). The test sends the same body under a fresh id and ten copies at once: Postgres decides, not a look-then-insert in the api.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A body that is not even JSON is a 401, not a 400.

Read the order again: the signature is checked before the body is parsed, so an unsigned non-JSON body is refused as *unauthenticated*. That is not an accident of ordering, it is the point: the api tells an anonymous caller nothing about what it does with a body, not even that it expects JSON. A signed body that is not a push is a 400, because whoever holds the secret is Gitea.

The rule for a public route: authenticate the bytes, then read them; bound the size before you do either; and never let a header you did not sign be the thing that makes two requests one.

5 · At least once, exactly once, and the second door

A webhook is at-least-once delivery at best, and Gitea's is worse: a failed delivery is *never retried* (SPEC §2a, measured in the learning test). So the api cannot be the only way a push is noticed, and a push noticed twice must be worked once. Both problems are solved in the same table.

FROM `packages/db/migrations/0009_git_events.sql`

```
-- E5.3a: the git peer's inbox. A row says "this branch of the repo now points at this commit". Rows come
-- through two doors: Gitea's push webhook (the api) and the git peer's own reconcile of its mirror, because
-- Gitea never retries a failed delivery (SPEC §2a). Not tenant data: the repo is the stack's one seed repo.
create table git_events (
  id          uuid primary key default gen_random_uuid(),
  ref         text not null check (ref ~ '^refs/heads/[A-Za-z0-9._-]{1,200}$'),
  before_sha  text not null check (before_sha ~ '^([0-9a-f]{40}|[0-9a-f]{64})$'),
  after_sha   text not null check (after_sha ~ '^([0-9a-f]{40}|[0-9a-f]{64})$'),
  -- X-Gitea-Delivery; null when the reconcile found the commit. The same delivery twice is one row.
  delivery_id text unique check (length(delivery_id) between 1 and 100),
  status      text not null default 'pending' check (status in ('pending', 'running', 'done', 'failed')),
  created_at  timestampz not null default now(),
  finished_at timestampz,
  check ((finished_at is not null) = (status in ('done', 'failed'))),
  -- The two doors meet here: whichever records a commit on a branch first wins, the other finds it taken.
  -- The database decides, not a "look, then insert" that two doors at the same instant would both pass.
  unique (ref, after_sha)
);
-- The git peer asks "what is waiting?" every second; a partial index keeps that cheap however many are done.
create index git_events_pending on git_events (created_at) where status = 'pending';

-- One flag: "a document was opened, reconcile soon". Requests coalesce: the peer clears it as it starts a
-- reconcile, so a burst of opens costs one fetch, and an open during a reconcile sets it again.
create table git_reconcile (
  id          boolean primary key default true check (id),
  requested   boolean not null default true
);
insert into git_reconcile default values;
```

Read it as a state machine and two keys. The status is `pending → running → done | failed`, and a `CHECK` ties `finished_at` to the terminal states, as lesson 3's jobs did. `delivery_id` is unique and nullable: the reconcile has none. `(ref, after_sha)` is unique and is *where the two doors meet*: whichever records a commit on a branch first wins, and the other finds it taken. The database decides. A "look, then insert" in the api would let two doors at the same instant both pass.

FROM `packages/db/src/index.ts`

```
gitStore: () => ({
  async record(input) {
    const e = GitEventInput.parse(input);
    // 'on conflict do nothing' with no target: EITHER unique key (delivery, or branch + commit) makes it a no-op.
    const result = await pool.query("insert into git_events (ref, before_sha, after_sha, delivery_id) values ($1, $2, $3, $4) on
    conflict do nothing returning id", [e.ref, e.before, e.after, e.deliveryId ?? null]);
    return result.rowCount === 1;
  },
  heads: async () =>
    new Map(await rows(
      z.object({ ref: z.string(), after_sha: z.string() }).transform((r): [string, string] => [r.ref, r.after_sha]),
      "select distinct on (ref) ref, after_sha from git_events order by ref, created_at desc, id desc",
      [],
    )),
  lastDone: async (ref) =>
    (await one(z.object({ after_sha: z.string() }), "select after_sha from git_events where ref = $1 and status = 'done' order by
    created_at desc, id desc limit 1", [ref]))?.after_sha,
  async claim(staleMs, maxResumes) {
    // coalesce(heartbeat_at, created_at): a row left running by a peer from before heartbeats (0021) is stale too.
    const stale = "status = 'running' and coalesce(heartbeat_at, created_at) < now() - make_interval(secs => $1::float8 / 1000)";
    // An event that took its peer down every time must end, or it would take every peer down in turn.
    await pool.query(`update git_events set status = 'failed', finished_at = now() where ${stale} and resumes >= $2`, [staleMs,
    maxResumes]);
    // ONE statement, 'skip locked': two peers never claim one event, and neither waits for the other. The row is
    // locked and its WHERE checked again on the newest version (read committed), so a stale event another peer
    // has just resumed (fresh heartbeat) is skipped, never resumed twice. 'status' on the right is the OLD one.
    // noon-wv8.3.2: an event handed back is passed by until its not_before.
    // noon-wv8.3.1.1: one event per branch at a time. The peer checks "already applied" once, as it starts; two of a
    // branch's events running side by side could land the older one's pages last. Claims take turns, so the
    // running row a claim looks for is one already committed, never one another claim is making at that instant.
    return inTurn("noon:git-claim", (client) => one(
      GitEventRow,
      `update git_events set status = 'running', heartbeat_at = now(), attempts = attempts + 1, resumes = resumes + (status =
      'running')::int
      where id = (select id from git_events e where (status = 'pending' and coalesce(not_before <= now(), true)
      and not exists (select 1 from git_events r where r.ref = e.ref and r.status = 'running')) or (${stale} and resumes < $2)
      order by created_at, id limit 1 for update skip locked) returning *`,
    ));
  },
});
```

```

        [staleMs, maxResumes],
        client,
    ));
},
heartbeat: async ({ id, attempt }) =>
    isId(id) && (await pool.query("update git_events set heartbeat_at = now() where id = $1 and status = 'running' and attempts = $2",
[id, attempt])).rowCount === 1,
    async finish({ id, attempt }, status, retryMs = 0) {
        if (!isId(id)) return;
        await pool.query(
            "update git_events set status = $2, finished_at = case when $2 = 'pending' then null else now() end, " +
            "not_before = case when $2 = 'pending' then now() + make_interval(secs => $4::float8 / 1000) end where id = $1 and status =
'running' and attempts = $3",
            [id, status, attempt, retryMs],
        );
    },
    async requestReconcile() {
        await pool.query("update git_reconcile set requested = true where not requested"); // no write, no row lock, when it is already
asked for
    },
    takeReconcileRequest: async () => (await pool.query("update git_reconcile set requested = false where requested")).rowCount === 1,

```

on conflict do nothing with *no target* means either unique key makes the insert a no-op, and returning id tells the caller which happened (rowCount === 1). claim() is lesson 3's pattern: one UPDATE with for update skip locked, so two peers never take one event and neither waits. And the reconcile flag is a *one-row table*: id boolean primary key default true check (id) can hold exactly one row, ever. Requests coalesce into it, and requestReconcile writes nothing when it is already set, so a burst of opens takes no row locks at all.

FROM apps/worker/src/git.ts

```

/**
 * The git peer's half of E5.3a (the api's webhook is the other door). It keeps a bare mirror of the repo
 * in its own directory (a volume: SPEC §2.15), records every branch that moved in Gitea but has no commit
 * event yet (Gitea never retries a failed delivery), and works through the events one at a time, each
 * in a worktree of its own that is removed however the job ends. What a commit means for a document
 * (ops through peer-client) is E5.3b's (push.ts): this hands it each generated page the commit touched,
 * and on request the page as it was before.
 */

```

FROM apps/worker/src/git.ts

```

/** What the reconcile records: every branch whose commit in the mirror is not its newest recorded one. A new branch starts from zeros. */
export function moved(mirror: ReadonlyMap<string, string>, recorded: ReadonlyMap<string, string>): Moved[] {
    return [...mirror].filter(([ref, sha]) => recorded.get(ref) !== sha).map(([ref, after]) => ({ ref, before: recorded.get(ref) ??
"0".repeat(after.length), after }));
}

```

FROM apps/worker/src/git.ts

```

async function reconcile(): Promise<number> {
    // Read BEFORE the fetch: a head recorded by then is at or behind the mirror. After it, a webhook landing
    // in between would be ahead of the mirror, and the "move" back to the mirror's tip recorded backwards.
    const recordedHeads = await store.heads();
    await fetch();
    const heads = parseHeads(await git("-C", mirror, "for-each-ref", "--format=%(refname) %(objectname)", "refs/heads/"));
    let recorded = 0;
    for (const event of moved(heads, recordedHeads)) if (await store.record(event)) recorded += 1;
    return recorded;
}

```

The reconcile is the second door: fetch the mirror, list its branches, and record every branch whose commit is not the newest one recorded, a new branch from zeros. It runs on a 30 s timer and *when a document is opened*: POST /documents/:id/session sets the flag, and the peer takes it **before** fetching, so an open that arrives during a fetch sets it again and is not lost. A new branch, which the webhook cannot tell from the registration push, is caught this way, and the e2e tests lean on it: they open the document to ask for a reconcile now.

FROM apps/worker/src/git.ts

```

async function processNext(): Promise<boolean> {
    const event = await store.claim(staleMs, maxResumes);
    if (!event) return false;
    const worktree = join(worktrees, event.id);
    const resumed = new AbortController(); // by another peer, after this one's beats went unseen
    const beat = setInterval(() => {
        void store.heartbeat(event).then((alive) => { if (!alive) resumed.abort(); }, () => undefined); // a missed beat: the next one
    }, Math.max(1, Math.floor(staleMs / 3)));
    try {
        if (!(await has(event.after))) {

```

```

    try {
      await fetch();
    } catch (err) {
      // Gitea is away: the event waits again rather than being lost (the reconcile would never re-record it).
      await store.finish(event, "pending", retryMs);
      log(`fetch failed, ${event.ref} ${event.after} waits: ${describeError(err)}`);
      return false;
    }
    // Fetched, and still not there: force-pushed away before we looked. Nothing to apply, ever.
    if (!(await has(event.after))) throw new Error(`commit ${event.after} of ${event.ref} is not in the repo`);
  }
  const lastDone = await store.lastDone(event.ref);
  const done = lastDone !== undefined && (await has(lastDone)) ? lastDone : undefined;
  if (await alreadyApplied(event, done)) {
    await store.finish(event, "skipped");
    log(`${event.ref} ${event.after} skipped: the branch was already applied up to ${done} ?? "${}"`);
    return true;
  }
  await mkdir(worktrees, { recursive: true });
  await git("-C", mirror, "worktree", "add", "--quiet", "--detach", worktree, event.after);
  const from = await diffBase(event, done);
  let base: Promise<string | undefined> | undefined;
  for (const page of await changedPages(event, from, worktree)) {
    if (resumed.signal.aborted) throw new Lost(`${event.ref} ${event.after}: another peer has resumed it`);
    await apply(event, page, async () => pageBase(await (base ??= baseCommit(event, from)), page.path));
  }
  await store.finish(event, "done");
} catch (err) {
  if (err instanceof Lost) {
    log(err.message);
    return true;
  }
  if (err instanceof WaitAgain) {
    // ponytail: the whole event waits, pages already applied included (applying again is safe: push-ops takes
    // the commit's journaled adds as its own, noon-91u). Ceiling: one retry per `retryMs` while the room stays read-only.
    await store.finish(event, "pending", retryMs);
    log(`${event.ref} ${event.after} waits: ${err.message}`);
    return false;
  }
  log(`${event.ref} ${event.after} failed: ${describeError(err)}`);
  await store.finish(event, "failed");
} finally {
  clearInterval(beat);
  await removeWorktree(worktree);
}
return true;
}

```

Each event is worked in its own *worktree* (`git worktree add --detach`), removed in `finally` by three steps that each tolerate the others' failure, and swept at start after a crash. A commit the mirror does not have is fetched; if the fetch fails, the event goes back to `pending` rather than `failed`, because the reconcile would never re-record it and a lost push is the one thing this design exists to prevent. Fetched and still missing means force-pushed away before we looked: nothing to apply, ever.

FROM `apps/worker/src/git.ts`

```

async start({ pollMs, reconcileMs, onAlive }) {
  // A worktree left by a peer that was killed mid-job: nothing may outlive its job, not even a crash.
  await rm(worktrees, { recursive: true, force: true });
  if (existsSync(join(mirror, "HEAD"))) await git("-C", mirror, "worktree", "prune").catch(() => undefined);
  let nextReconcile = 0;
  async function tick(): Promise<void> {
    try {
      // Taken BEFORE the fetch: a document opened while it runs asks again, and is not lost.
      if ((await store.takeReconcileRequest()) || Date.now() >= nextReconcile) {
        nextReconcile = Date.now() + reconcileMs;
        await reconcile();
      }
    } catch (err) {
      nextReconcile = Date.now() + Math.min(reconcileMs, 10_000); // Gitea away: again soon, not every tick
      log(`reconcile failed: ${describeError(err)}`);
    }
    try {
      for (let n = 0; n < EVENTS_PER_TICK && (await processNext()); n += 1);
      onAlive?();
    } catch (err) {
      log(`inbox unreadable: ${describeError(err)}`);
    }
  }
  let running: Promise<void> | undefined;
  const beat = (): void => void (running ??= tick().finally(() => (running = undefined)));
  beat();
}

```

```

    const timer = setInterval(beat, pollMs);
    return {
      async stop() {
        clearInterval(timer);
        await running;
      },
    };
  },
};
};
}

```

Two idioms in the tick. `running ??= tick().finally(() => (running = undefined))` is "one tick at a time" in one line: `??=` assigns only when the left side is nullish, so a beat that arrives while a tick runs does nothing, and the `finally` clears the slot. And `for (let n = 0; n < EVENTS_PER_TICK && (await processNext()); n += 1);` is a loop with an empty body: the work is in the condition, and it stops at the first "nothing waiting" or after twenty.

FROM `apps/worker/src/git.ts`

```

/** A page as the job's worktree holds it. O_NOFOLLOW: a pushed symlink named like a page must not read /proc or the worker's files. */
async function readPage(file: string): Promise<{ tsx: string } | { refused: "deleted" | "not_a_file" | "too_large" }> {
  let handle;
  try {
    handle = await open(file, constants.O_RDONLY | constants.O_NOFOLLOW);
  } catch (err) {
    const code = (err as { code?: unknown }).code;
    if (code === "ENOENT") return { refused: "deleted" };
    if (code === "ELOOP" || code === "EMLINK") return { refused: "not_a_file" }; // the symlink itself (EMLINK on some BSDs)
    throw err;
  }
  try {
    const stat = await handle.stat();
    if (!stat.isFile()) return { refused: "not_a_file" }; // a submodule checks out as a directory
    if (stat.size > MAX_PAGE_BYTES) return { refused: "too_large" };
    return { tsx: await handle.readFile("utf8") };
  } finally {
    await handle.close();
  }
}

```

FROM `apps/worker/src/git.int.test.ts`

```

test("the webhook and the reconcile meet at one event for one push, whichever is first", async () => {
  const { peer: p } = peer();
  await p.reconcile();
  const before = await git(work, "rev-parse", "HEAD");
  const first = await commit({ "a.txt": "1\n" }, "one");
  // The webhook was first: the reconcile finds main already where it is.
  expect(await t.db.gitStore().record({ ref: "refs/heads/main", before, after: first, deliveryId: "d-1" })).toBe(true);
  expect(await p.reconcile()).toBe(0);
  // The reconcile was first: the late webhook is a duplicate.
  const second = await commit({ "a.txt": "2\n" }, "two");
  expect(await p.reconcile()).toBe(1);
  expect(await t.db.gitStore().record({ ref: "refs/heads/main", before: first, after: second, deliveryId: "d-2" })).toBe(false);
  expect((await events()).filter((e) => e.after_sha === first || e.after_sha === second)).toHaveLength(2);
});

test("missed non-tip push is folded into the next event: its page still reaches apply (noon-wv8.3.1)", async () => {
  const { seen, apply } = collect();
  const { peer: p } = peer(apply);
  await p.reconcile();
  while (await p.processNext());
  const lost = await commit({ [pagePath(DOC)]: "v1\n" }, "its delivery was lost (the api was restarting)");
  const delivered = await commit({ [pagePath(OTHER)]: "w1\n" }, "its delivery arrived");
  expect(await t.db.gitStore().record({ ref: "refs/heads/main", before: lost, after: delivered, deliveryId: "d-2" })).toBe(true);
  expect(await p.reconcile()).toBe(0); // the tip is recorded: the lost push never gets an event of its own
  expect(await p.processNext()).toBe(true);
  expect(seen.map((s) => s.page).sort((a, b) => a.documentId.localeCompare(b.documentId))).toEqual([
    { documentId: DOC, path: pagePath(DOC), tsx: "v1\n" },
    { documentId: OTHER, path: pagePath(OTHER), tsx: "w1\n" },
  ]);
  expect((await events()).map((e) => e.status)).toEqual(["done", "done"]);
});

```

FROM `apps/worker/src/git.int.test.ts`

```

test("while Gitea is away an event waits instead of being lost, and is worked on once it is back", async () => {
  const { seen, apply } = collect();
  const { peer: p, logs } = peer(apply);

```

```

await p.reconcile();
while (await p.processNext());
const moved = join(root, "away.git");
const pushed = await commit({ [pagePath(DOC)]: "v2\n" }, "pushed, then Gitea went down");
await t.db.gitStore().record({ ref: "refs/heads/main", before: await git(work, "rev-parse", "HEAD~1"), after: pushed, deliveryId: "d"
});
await exec("mv", [origin, moved]);
expect(await p.processNext()).toBe(false);
expect((await events()).at(-1)?.status).toBe("pending");
expect(logs.join("\n")).toMatch(/fetch failed/);
expect(await p.processNext()).toBe(false); // noon-wv8.3.2: passed by until its retry, not claimed (and fetched) again at once
expect(logs.filter(line => line.includes("fetch failed"))).toHaveLength(1);
await exec("mv", [moved, origin]);
await expect.poll(() => p.processNext(), { timeout: 5000 }).toBe(true);
expect(seen.map((s) => s.page)).toEqual([{ documentId: DOC, path: pagePath(DOC), tsx: "v2\n" }]);
});

test("E6.1b: a page whose document is read-only waits instead of failing, and is worked on once the room can save", async () => {
  let readOnly = true;
  const seen: string[] = [];
  const { peer: p, logs } = peer((_event, page) => {
    if (readOnly) return Promise.reject(new WaitAgain("document_read_only"));
    seen.push(page.path);
    return Promise.resolve();
  });
  await p.reconcile();
  while (await p.processNext());
  seen.length = 0;
  readOnly = true;
  await commit({ [pagePath(DOC)]: "v2\n" }, "pushed while Postgres was away from the room");
  await p.reconcile();
  expect(await p.processNext()).toBe(false);
  expect((await events()).at(-1)?.status).toBe("pending");
  expect(logs.join("\n")).toMatch(/document_read_only/);
  readOnly = false;
  await expect.poll(() => p.processNext(), { timeout: 5000 }).toBe(true); // once its retry time has come (noon-wv8.3.2)
  expect((await events()).at(-1)?.status).toBe("done");
  expect(seen).toEqual([pagePath(DOC)]);
  onlyTheMirror(await worktreesLeft());
});

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
two unique keys and on conflict do nothing as the dedupe	the same SQL; Spring Integration's <code>IdempotentReceiver</code> if you must	the same SQL
for update skip locked to claim	the same, as a native query	the same
a one-row table: id boolean primary key default true check (id)	the same trick; or a singleton row by convention	the same
running ??= tick().finally(...)	an <code>AtomicBoolean</code> guard, or a single-thread executor	an <code>asyncio.Lock</code> , or a task you check <code>done()</code> on
open(path, O_RDONLY O_NOFOLLOW) from node:fs/promises	<code>Files.newByteChannel(p, NOFOLLOW_LINKS)</code>	<code>os.open(p, os.O_RDONLY os.O_NOFOLLOW)</code>
a worktree per job, removed in finally, swept at start	a temp dir per job with a shutdown hook and a startup sweep	the same

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A pushed symlink named like a page could have read `/proc/self/environ`.

The git peer reads each changed page from the job's worktree. A commit can contain a symlink at `src/pages/noon-<id>.tsx` pointing anywhere: `/proc/self/environ` holds the peer's own environment, including the Gitea token. A plain `readFile` would follow it. The page is opened with `O_NOFOLLOW`, which fails with `ELLOOP` on a symlink; `stat().isFile()` refuses a directory (a submodule checks out as one); and the size is checked against `parse's` cap before the read. The refusal has a name the banner shows: `not_a_file`.

FROM `apps/worker/src/git.int.test.ts`

```
test("a pushed symlink named like a page is refused, never followed out of the worktree", async () => {
  const { seen, apply } = collect();
  const { peer: p } = peer(apply);
  await p.reconcile();
  while (await p.processNext());
  writeFileSync(join(root, "secret"), "the worker's secrets\n");
  mkdirSync(join(work, "src/pages"), { recursive: true });
  symlinkSync(join(root, "secret"), join(work, pagePath(DOC)));
  await commit({}, "symlink");
  await commit({ [pagePath(OTHER)]: "" }, "and a real one");
  await p.reconcile();
  while (await p.processNext());
  expect(seen.map((s) => s.page).sort((a, b) => a.documentId.localeCompare(b.documentId))).toEqual([
    { documentId: DOC, path: pagePath(DOC), refused: "not_a_file" },
    { documentId: OTHER, path: pagePath(OTHER), tsx: "" },
  ]);
  onlyTheMirror(await worktreesLeft());
});
```

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A force-push back to an old commit reads as "known", and the comment says so.

Keying on `(ref, after_sha)` means a branch moved $A \rightarrow B \rightarrow A$ records nothing the second time it points at A: the row for `(ref, A)` exists. This is a ponytail with a named ceiling (such reverts go unseen) and a named upgrade (key on `(ref, before, after)`). The reason it is acceptable today is what the loop does with events: a page that is already what the document holds produces no ops, so a missed revert is a missed no-op until the next real push.

The distributed-systems shape, in one paragraph: a producer that may deliver zero or one times; a receiver that writes an idempotent row keyed on the *content*, not the delivery; a second source of truth (the mirror) polled on a timer and on demand, writing through the same key; a claim that is one atomic update; and a failure mode ("the source is unreachable") that returns the work to the queue rather than ending it. Every piece is in Postgres, and none of it is in application memory.

6 · A push becomes ops: git is a peer

Keystone 2 says every change to a document goes through the room. The git peer honours it literally: it joins the document through `@noon/peer-client` like a browser tab, with a session token whose actor is `git`, and submits ops. The room stamps every op with the actor and the commit as its run, and the room may refuse any op, one by one. A push that wrote to Postgres directly would be a second write path, and the whole of epic 2 (ordering, budgets, presence, the journal to come in epic 6) would not apply to it.

FROM `apps/worker/src/push.ts`

```
/**
 * E5.3b (F16a): a pushed generated page becomes ops on its open document. The git peer joins the
 * document through @noon/peer-client like a browser tab and the AI do (keystone 2, the ONE write path),
 * with a session whose actor is 'git': the room stamps every op with it, and with the commit as its run.
 *
 * Only the document's own branch ('noon-<id>', the one the sandbox works on and Ship pushes) speaks for
 * it: the same file on main or on a feature branch would otherwise land twice, once per branch.
 *
 * Out of shape, another page's root, an id the page held before: a conflict, and NOTHING is sent (E5.4
 * shows it: keepConflict). Whatever the room then refuses op by op (a remove on the canvas won the race) is counted.
 */
export type PushOutcome =
  | { kind: "skipped"; why: "other_branch" | "no_document" | "shipped" }
  | { kind: "conflict"; reason: ConflictReason; detail: string }
  | { kind: "applied"; ops: number; refused: number };
```

FROM apps/worker/src/push.ts

```
const peer = connectPeer({
  manifest,
  // Named by the commit and the op itself: the same push applied again sends its ops under their first ids, and the
  // room answers a repeat (a set_prop the canvas has changed since, included) with its first answer, not a second edit.
  mintOpId: (op) => stableOpId(`${event.after}:${page.documentId}:${JSON.stringify(op)}`),
  ...(WebSocketImpl ? { WebSocketImpl } : {}),
  // 'sub' must be a uuid: the event's id. The commit rides as the run, so every op names the commit it came from.
  session: async () => ({
    wsUrl: await roomUrl(sessions, page.documentId),
    token: signSessionToken({ userId: event.id, orgId, documentId: page.documentId, secret: sessions.secret, ttlSeconds: 60, actor: {
kind: "git", runId: event.after } }},
  }},
});
try {
  await whenLive(peer, connectTimeoutMs);
  // The CONFIRMED document: the base of the diff must be what the room holds, never a guess.
  // Asked once the room is live, so an add journaled before this peer joined is in the answer.
  const alreadyAdded = await pushedNodeIds(page.documentId, event.after);
  const result = pushOps({ base: baseDoc?.ok ? baseDoc.doc : undefined, target: parsed.doc, current: peer.confirmed, earlierIds:
before.earlierIds, alreadyAdded });
  if (!result.ok) return { kind: "conflict", reason: result.reason, detail: result.detail };
  const settled = [];
  for (const op of result.ops) {
    const submitted = peer.submit(op);
    // E6.1b: the room cannot save. Not a conflict (the page is fine) and not a failure (the push would be
    // lost): the event waits. Ops of this page already sent die with the peer below, unapplied.
    if (!submitted.ok && submitted.reason === "read_only") throw new WaitAgain("document_read_only");
    // The replica refused it locally: the whole page was checked against this very document, so this is a bug; say so.
    if (!submitted.ok) throw new Error(`the git peer's own op was refused: ${submitted.reason}`);
    settled.push(submitted.settled);
  }
  const outcomes = await within(settleTimeoutMs, "sync_timeout", Promise.all(settled));
  return { kind: "applied", ops: result.ops.length, refused: outcomes.filter((outcome) => !outcome.ok).length };
} finally {
  peer.close(); // an op still unanswered is settled 'connection_closed'
}
};
```

Read `current: peer.confirmed`. The base of the diff is what the room holds, never the optimistic tree and never a copy in Postgres that may be behind the room. And read `peer.close()` in `finally`: an op still unanswered is settled `connection_closed`, so nothing awaits for ever. The session's `sub` must be a uuid, so it is the event's id; the commit rides as `runId`. That is why the e2e test can assert, from the frames the canvas received, that the op came from `{ kind: "git", runId: <the commit> }`.

The diff is three-way. The engineer cloned the branch at some commit, edited the page, pushed. Meanwhile the canvas kept editing. Diffing the *pushed file* against the *document* would treat every canvas edit since the file was generated as something the engineer undid. So: `base` = the page at the commit before the push, `target` = the pushed page, and the ops are what changed between them, replayed on the document as it is *now*.

FROM apps/worker/src/push-ops.ts

```
/**
 * E5.3b: what a push means for the open document, as ops. A THREE-WAY diff: 'base' is the page the
 * engineer started from (the file at the commit before the push), 'target' the page they pushed, and
 * the ops are what they changed, replayed on the document as it is NOW. Diffing the pushed file
 * against the document instead would undo every canvas edit made since that file was generated.
 *
 * Minimal: a prop that did not change is not sent, a node that kept its place among its siblings is
 * not moved (the longest run of siblings still in order stays), and a removed subtree is one remove.
 * Deterministic: everything is walked in tree order and props in name order, never in key order.
 *
 * Each op is tried on a copy of the document as it is made. One that would change nothing there (its
 * node or its parent was removed on the canvas meanwhile, the value is already set, the move would make
 * a cycle) is not sent: the same result the room would reach, as keystone 4's concurrent edits do.
 */
```

FROM apps/worker/src/push-ops.ts

```
export function pushOps({ base, target, current, earlierIds, alreadyAdded = new Set() }: {
  /** The page before the push, if it was there and in shape. Missing: the document is the base (the push is taken as the whole page). */
  base: Doc | undefined;
  target: Doc;
  /** The document now: 'peer.confirmed', never the optimistic one. */
  current: Doc;
  /** Every node id the page has held in the repo's history up to the base: an id that is gone from the base was removed. */
  earlierIds: ReadonlySet<string>;
  /**
```

```

* noon-91u: the ids THIS push (its commit) already added to the document, from the journal: the push is being
* applied again (its git peer was killed halfway). Each is its own add, not a re-use: it is not refused and not
* added twice, and one the canvas removed since is not brought back.
*/
alreadyAdded?: ReadonlySet<string>;
}): PushOps {
  if (target.rootId !== current.rootId) return { ok: false, reason: "root_mismatch", detail: `the file's page is ${target.rootId}, the
  document's is ${current.rootId}` };
  const from = base?.rootId === current.rootId ? base : current;
  // Keystone 4: an id that was removed is never added again, or an edit meant for the dead node would land
  // on the new one. A new id the document already holds is the same mistake, seen from the other side.
  for (const id of preorder(target)) {
    const was = nodeOf(from, id);
    if (!was && !alreadyAdded.has(id) && (earlierIds.has(id) || nodeOf(current, id))) return { ok: false, reason: "reused_node_id",
    detail: `${id} was part of this document before` };
    // No op changes what a node IS; a remove and an add would re-use its id.
    if (was && was.component !== nodeOf(target, id)?.component) return { ok: false, reason: "component_changed", detail: `${id} was a
    ${was.component}` };
  }
}

```

Before any op is made, two whole-page refusals. `root_mismatch` : the file's page is another document's. `reused_node_id` : keystone 4 says an id that was removed is never added again, or an edit meant for the dead node lands on the new one; "removed" is decided against *every id the page ever held in the repo's history* (`earlierIds` , from `git log -p`) and against the document now. `component_changed` : no op changes what a node *is*; a remove and an add would re-use its id.

FROM `apps/worker/src/push-ops.ts`

```

const working = structuredClone(current);
const ops: Op[] = [];
const emit = (op: Op): void => { if (applyOpInto(working, op)) ops.push(op); };
/** The final index that puts a node right after `anchor` among `parentId`'s children (first, with no anchor). */
const after = (parentId: string, nodeId: string, anchor: string | undefined): number => {
  const siblings = (nodeOf(working, parentId)?.children ?? []).filter((id) => id !== nodeId);
  return anchor === undefined ? 0 : siblings.indexOf(anchor) + 1;
};

// 1. Adds and moves, parents before children, so a node's new parent is in place before it arrives.
for (const parentId of preorder(target)) {
  const children = nodeOf(target, parentId)?.children ?? [];
  const stays = inOrder(children, nodeOf(from, parentId)?.children ?? [], (id) => nodeOf(from, id)?.parentId === parentId);
  let anchor: string | undefined;
  for (const id of children) {
    const node = nodeOf(target, id);
    if (!node) continue;
    // Already added by this push: where it is now (and whatever the canvas did to it since) stands.
    if (!nodeOf(from, id)) {
      if (!alreadyAdded.has(id)) emit({ type: "add_node", nodeId: id, parentId, index: after(parentId, id, anchor), component:
      node.component, props: { ...node.props } });
      } else if (!stays.has(id)) emit({ type: "move_node", nodeId: id, newParentId: parentId, index: after(parentId, id, anchor) });
      if (nodeOf(working, id)?.parentId === parentId) anchor = id;
    }
  }
}

// 2. Props of the nodes both pages hold (an added node brought its own).
for (const id of preorder(target)) {
  const before = nodeOf(from, id)?.props;
  const now = nodeOf(target, id)?.props;
  if (!before || !now || id === target.rootId) continue;
  for (const key of [...new Set([...Object.keys(before), ...Object.keys(now)])].sort()) {
    const value = Object.hasOwn(now, key) ? now[key] : undefined;
    if ((Object.hasOwn(before, key) ? before[key] : undefined) !== value) emit({ type: "set_prop", nodeId: id, key, value: value ?? null
  });
  }
}

// 3. Removes, last: what was moved out of a removed node is out by now. Parents first, so a removed
// subtree is one op (its descendants are gone from `working` by the time they come up).
for (const id of preorder(from)) if (id !== from.rootId && !nodeOf(target, id)) emit({ type: "remove_node", nodeId: id });
return { ok: true, ops };
}

```

Then the ops, in three passes over a *working copy* (`structuredClone` : a deep copy of plain data, built into modern JavaScript). Each op is tried on the copy with `applyOpInto` , the mutating applicer from lesson 2, and only an op that changed the copy is kept: one whose node was removed on the canvas meanwhile, or whose value is already set, is dropped, which is the result the room would reach anyway. Adds first, parents before children, each placed after the previous sibling that is in place (`anchor`). Then props, in sorted key order, so the ops are the same on every run. Then removes, parents first, so a removed subtree is one op. Moves use the longest run of siblings still in their old order, which is a longest increasing subsequence in $O(n \log n)$ with a binary search written as `(lo + hi) >> 1` .

FROM apps/worker/src/push-ops.ts

```
/**
 * The siblings that need no move: of those that were already under this parent, the longest run that is
 * still in the old order (a longest increasing subsequence of their old positions,  $O(n \log n)$ ). Among
 * runs of the same length it keeps the one that ends lowest, the same one every time.
 */
function inOrder(now: readonly string[], before: readonly string[], wasHere: (id: string) => boolean): Set<string> {
  const position = new Map(before.map((id, i) => [id, i]));
  const ids = now.filter((id) => wasHere(id) && position.has(id));
  const tails: number[] = []; // tails[k]: index in `ids` of the smallest end of a run of length k+1
  const previous: number[] = [];
  ids.forEach((id, i) => {
    const p = position.get(id) ?? 0;
    let lo = 0;
    let hi = tails.length;
    while (lo < hi) {
      const mid = (lo + hi) >> 1;
      if ((position.get(ids[tails[mid] ?? 0] ?? "") ?? 0) < p) lo = mid + 1;
      else hi = mid;
    }
    previous[i] = lo > 0 ? (tails[lo - 1] ?? -1) : -1;
    tails[lo] = i;
  });
  const stays = new Set<string>();
  for (let i = tails.at(-1) ?? -1; i >= 0; i = previous[i] ?? -1) stays.add(ids[i] ?? "");
  return stays;
}
```

FROM apps/worker/src/git.ts

```
async function pageBase(commit: string | undefined, path: string): Promise<PageBase> {
  if (commit === undefined) return { tsx: undefined, earlierIds: new Set() };
  // "<mode> blob <object> <size>\t<path>": only a regular file within parse's cap is read (a symlink's blob is its target's name).
  const entry = /^100(?:644|755) blob ([0-9a-f]{40,64}) +(\d+)\t/u.exec(await git("-C", mirror, "ls-tree", "-l", "-z", commit, "--", path));
  const tsx = entry?.[1] !== undefined && Number(entry[2]) <= MAX_PAGE_BYTES ? await git("-C", mirror, "cat-file", "blob", entry[1]) : undefined;
  // --full-history: an id added and removed again on a side branch that a merge left out still counts.
  // ponytail: the whole history of one file, held in memory (cli's buffer cap); upgrade: stream it.
  const log = await git("-C", mirror, "log", "--full-history", "--no-renames", "--no-ext-diff", "--no-textconv", "--no-color", "-p", "-U0", "--format=", commit, "--", path);
  return { tsx, earlierIds: nodeIdsIn(log) };
}
```

FROM apps/worker/src/push-ops.test.ts

```
test("the push replays what the ENGINEER changed on the document as it is now: canvas edits since stay", () => {
  // Since the file was generated, the canvas renamed A, added d and removed c.
  const now = [{ type: "set_prop", nodeId: "a", key: "title", value: "renamed on the canvas" }, add("d", "root", "Button", { label: "D" })], { type: "remove_node", nodeId: "c" }] satisfies Op[];
  const current = now.reduce(applyOp, page);
  // The engineer changed b's label, edited c (gone now) and put a new node first.
  const pushed = [{ type: "set_prop", nodeId: "b", key: "label", value: "from git" }, { type: "set_prop", nodeId: "c", key: "value", value: "edited" }, add("g", "root", "Text", { value: "G" }, 0)] satisfies Op[];
  const result = ops(page, pushed.reduce(applyOp, page), current);
  expect(result).toEqual([
    { type: "add_node", nodeId: "g", parentId: "root", index: 0, component: "Text", props: { value: "G" } },
    { type: "set_prop", nodeId: "b", key: "label", value: "from git" }, // c's edit is dropped: a remove beats a concurrent edit
  ]);
  const after = result.reduce(applyOp, current);
  expect(after.nodes["a"]?.props).toEqual({ title: "renamed on the canvas" });
  expect(after.nodes["root"]?.children).toEqual(["g", "a", "b", "d"]);
});
```

FROM apps/worker/src/push.int.test.ts

```
test("the push replays the engineer's change on the document as it is NOW: canvas edits made since stay", async () => {
  await edit(button("b1", "Before"));
  await edit(button("b2", "Other"));
  const base = doc;
  await push(base);
  // On the canvas, after the file was pushed: b2 is renamed and b1 removed.
  await edit(setLabel("b2", "renamed on the canvas"));
  await edit({ type: "remove_node", nodeId: "b1" });
  // The engineer, from the old file, edits b1 (gone now) and adds b3.
  await push([setLabel("b1", "too late"), button("b3", "New")].reduce(applyOp, base));
  expect(outcomes.at(-1)).toEqual({ kind: "applied", ops: 1, refused: 0 });
  expect(await person.next("op", (m) => m.actor.kind === "git")).toMatchObject({ op: { type: "add_node", nodeId: "b3" } });
});
```

<pre>expect(gitOps()).toHaveLength(1); // no revert of the rename, no b1 brought back });</pre>		
TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>structuredClone(doc)</code> : a deep copy of plain data	no built-in; copy constructors, or Kryo	<code>copy.deepcopy</code>
<code>satisfies Op[]</code> : check a literal against a type without widening it	no equivalent; the declared type IS the widening	no equivalent
<code>tails.at(-1); (lo + hi) >> 1</code>	<code>list.get(list.size() - 1); >>> 1</code> for the unsigned midpoint	<code>tails[-1]; (lo + hi) // 2;</code> <code>bisect</code>
<code>for (let id = stack.pop(); id !== undefined; id = stack.pop())</code>	<code>while (!stack.isEmpty()) { var id = stack.pop(); }</code>	<code>while stack: id = stack.pop()</code>
the git peer as a WebSocket client of our own sync server	the same: a service that is a client of another	the same
a JWT with <code>actor: { kind: "git", runId: commit }</code>	a service account with claims	the same

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Diffing the pushed file against the document would have undone every canvas edit since the file was generated.

This is the rejected design, and the second drill plants it. The unit test builds the case: since the page was generated the canvas renamed A, added D and removed C; the engineer, from the old file, relabelled B, edited C and added G first. Two-way against the document sends: add G, *re-add C, rename A back*, relabel B, *remove D*. Three-way sends: add G, relabel B; C's edit is dropped because a remove beats a concurrent edit, the same rule the room applies to two browser tabs. The property test then runs three hundred random pages and random edits and checks the ops rebuild the pushed page exactly and that the same inputs give the same ops.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Ship's own pushes would come back as events and undo the canvas edits that raced them.

Named in E5.3b's commit as a hand-off, closed in E5.5. When Ship pushes the page it generated from the document, the webhook fires and the git peer sees a push on the document's branch. Diffed, that push says "the label went from *Before* to *Shipped*", which the room already holds; but a canvas edit made *while* Ship ran would be reverted by it. So Ship records its commit on its job **before** pushing, and the git peer asks `shippedCommit(sha)` before it reads or opens anything. The order is the whole fix: recorded after the push, the webhook could win the race.

FROM `apps/worker/src/push.int.test.ts`

```
test("a commit Ship made is skipped: a canvas edit that raced the ship stays (E5.5)", async () => {
  await edit(button("b1", "Before"));
  await push(doc); // the branch as it was: the last ship, or an engineer's push
  await edit(setLabel("b1", "Shipped"));
  const shipped = doc; // what Ship read and generated
  await edit(setLabel("b1", "edited on the canvas while shipping"));
  // Ship records its commit on its job BEFORE it pushes, so the commit is known by the time the push is seen.
  const org = await t.createOrg("ship");
  const workspace = await t.db.forOrg(org.id).createWorkspace({ name: "w" });
  const document = await t.db.forOrg(org.id).createDocument({ workspaceId: workspace.id, title: "d" });
  await push(shipped, BRANCH, async (sha) => {
    const job = (await t.rawQuery("insert into jobs (org_id, document_id, queue, status, started_at, input) values ($1, $2, 'ship', 'running', now(), '{}') returning id", [org.id, document?.id])) as { rows: [{ id: string }] };
    await t.db.jobStore().report({ queue: "ship", jobId: job.rows[0].id, orgId: org.id, { commit: sha, pr: null } }); // as ship.ts's recordCommit does
  });
  expect(outcomes.at(-1)).toEqual({ kind: "skipped", why: "shipped" });
  // Diffed, "Before" -> "Shipped" would have been replayed onto the room: the racing edit, undone.
  expect(gitOps()).toEqual([]);
});
```

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Only the document's own branch speaks for it.

The same page on `main` and on `noon/<id>` would land twice, once per branch, and a feature branch of the engineer's would edit a document they were only experimenting on. So a page is applied only from `refs/heads/noon/<its document>`: the branch the sandbox works on and Ship pushes. Everything else is `skipped: other_branch`, and a skipped page says nothing about the document, so it neither sets nor clears a conflict (section 7).

The rule, in one line: when two parties edit the same thing from different starting points, diff each against *its own* starting point and replay onto the current state. That is what git itself does in a merge, what lesson 2's room does with two tabs, and what a push-becomes-ops has to do or it is a revert engine.

7 · A refusal that is visible: the conflict banner

Section 6 already refused an out-of-shape page whole: nothing sent, so the room's tree and its `seq` cannot move. E5.4 makes the refusal visible without changing that. The git peer records it; the canvas reads it every two seconds and shows a `role="alert"` above the editor: commit, file, and why. Editing is not blocked. The e2e proves "nothing moved" by hashing the room's tree as a fresh joiner is welcomed with it, before and after the push.

FROM `packages/db/migrations/0010_document_conflicts.sql`

```
-- E5.4 (F16b): the newest push to a document's branch that the git peer refused, which the canvas shows as a
-- banner. One row per document: a later refused push replaces it, a later applied one removes it. Nothing
-- else is kept: the document itself was not touched, and git holds the commit.
create table document_conflicts (
  document_id uuid primary key references documents (id) on delete cascade,
  commit_sha text not null check (commit_sha ~ '^[0-9a-f]{40}|[0-9a-f]{64}$'),
  file text not null check (length(file) between 1 and 300),
  reason text not null check (reason ~ '^[a-z]{1,40}$'),
  detail text not null check (length(detail) <= 300),
  created_at timestamptz not null default now()
);
```

FROM `packages/db/src/index.ts`

```
async recordConflict(documentId, input) {
  if (!isId(documentId)) return;
  const c = ConflictInput.parse(input);
  // Selected from documents: a document deleted meanwhile inserts nothing instead of failing on the foreign key.
  // The rejected push goes into the document's org's audit trail in the same statement (F26). ponytail: the actor
  // is "git" and the commit, not the person who pushed; ceiling: the trail cannot name them; upgrade: keep the
  // webhook's pusher on git_events and copy it here.
  // noon-dtf.4.1: once per (document, commit). A resumed event (its peer died after this write, before finishing
  // it) refuses the same commit again, and the trail must not list one rejected push twice. ponytail: the guard
  // reads the org's rows through audit_log_org; upgrade: a partial index on (document_id, detail -> 'commit').
  await pool.query(
    "with c as (insert into document_conflicts (document_id, commit_sha, file, reason, detail) select id, $2, $3, $4, $5 from
documents where id = $1 " +
    "on conflict (document_id) do update set commit_sha = excluded.commit_sha, file = excluded.file, reason = excluded.reason,
detail = excluded.detail, created_at = now() returning document_id) " +
    `insert into audit_log ${AUDIT_COLUMNS} select d.org_id, 'git', null, null, 'push_rejected', d.id,
jsonb_build_object('commit', $2::text, 'file', $3::text, 'reason', $4::text) from c join documents d on d.id = c.document_id ` +
    "where not exists (select 1 from audit_log a where a.org_id = d.org_id and a.document_id = d.id and a.action = 'push_rejected'
and a.detail -> 'commit' = $2)",
    [documentId, c.commit, c.file, c.reason, c.detail],
  );
},
```

One row per document, by primary key. `insert ... select ... from documents where id = $1` inserts nothing for a document deleted meanwhile, instead of failing on the foreign key. `on conflict (document_id) do update set ... = excluded.*` is Postgres's upsert: `excluded` is the row that *would* have been inserted. A later refused push replaces the conflict; a later applied one deletes it; a skipped one changes nothing.

FROM `packages/db/src/index.ts`

```
// `detail` is parse's words about a file an engineer wrote: capped here, not refused (the conflict must still be shown).
const ConflictInput = Conflict.omit({ at: true, detail: true }).extend({ detail: z.string().transform((d) => d.slice(0, 300)) });
```

`detail` is capped by a `Zod .transform`, not refused: parse's words about an engineer's file may be long, and a conflict must always be shown. That is a trust-boundary decision made in the right direction: what is validated (the reason, the sha, the file length) is what a stranger could exploit; what is only trimmed is what would merely be inconvenient.

FROM packages/contracts/src/index.ts

```
// --- Conflicts (F16b) -----
/**
 * Why a pushed generated page was not applied: codegen's parse reasons, push-ops' history checks, and the
 * git peer's own refusals of the file. The worker assigns its reasons to this type, so a reason added there
 * and not here stops compiling (and the canvas's Record of sentences stops compiling in turn).
 */
export const ConflictReason = z.enum([
  // codegen (generate's rules, applied to the parsed tree)
  "unknown_prop", "wrong_prop_type", "missing_required_prop", "malformed_doc", "unknown_component", "reserved_component",
  "parent_takes_no_children",
  // parse (the fixed shape)
  "too_large", "too_deep", "syntax_error", "extra_statement", "hook", "second_export", "not_page_component", "bad_import", "spread",
  "conditional", "map",
  "non_literal_prop", "text_child", "expression_child", "not_an_element", "missing_node_id", "bad_node_id", "duplicate_node_id",
  "duplicate_prop",
  // push-ops (the document's history)
  "root_mismatch", "reused_node_id", "component_changed",
  // the git peer (the file itself)
  "deleted", "not_a_file",
]);
export type ConflictReason = z.infer<typeof ConflictReason>;
/**
 * A push to the document's branch that changed nothing, as the canvas shows it (F16b). `commit` and `file`
 * come from an engineer's push: the canvas renders them as TEXT, never as markup. `detail` is parse's own
 * words (a line number, what it met), capped.
 */
export const Conflict = z.strictObject({
  commit: z.string().regex(/^([0-9a-f]{40}|[0-9a-f]{64})$/u),
  file: z.string().min(1).max(300),
  reason: ConflictReason,
  detail: z.string().max(300),
  at: z.iso.datetime(),
});
export type Conflict = z.infer<typeof Conflict>;
/** GET /documents/:id/conflict. null: the newest push to the document's branch was applied, or there was none. */
export const DocumentConflict = z.strictObject({ conflict: Conflict.nullable() });
export type DocumentConflict = z.infer<typeof DocumentConflict>;
```

One vocabulary, checked by the compiler. `ConflictReason` is one Zod enum listing codegen's reasons, parse's, push-ops' and the git peer's own.

`PushOutcome`'s reason is typed with it, so a reason added in the worker without a contract entry stops compiling. And the canvas's sentences are a `Record<ConflictReason, string>`, so a reason added to the contract without a sentence stops compiling in turn. Two compile errors are how a new refusal announces itself to two teams.

FROM apps/web/src/conflict.ts

```
/**
 * Why a pushed file was not applied, in the user's words. A Record over the WHOLE contract, like reasons.ts:
 * a reason added there stops this file compiling until someone has written its sentence.
 */
const SENTENCES: Record<ConflictReason, string> = {
```

FROM apps/web/src/conflict.ts

```
/** What the banner says, as plain strings: the caller renders them as TEXT (commit and file are an engineer's). */
export function conflictWords(conflict: Conflict): { commit: string; file: string; why: string } {
  return { commit: conflict.commit, file: conflict.file, why: conflict.detail === "" ? SENTENCES[conflict.reason] :
    `${SENTENCES[conflict.reason]} (${conflict.detail}` }
}
```

FROM apps/web/src/ConflictBanner.tsx

```
export function ConflictBanner({ documentId }: { documentId: string }) {
  const [conflict, setConflict] = useState<Conflict | null>(null);
  useEffect(() => {
    let stopped = false;
    let timer: ReturnType<typeof setTimeout> | undefined;
    const tick = async (): Promise<void> => {
      const next = await readConflict(documentId).catch(() => undefined); // a failed look is tried again
      if (stopped || next === "gone") return; // the canvas says so itself when the document goes
      if (next !== undefined) setConflict(next);
      timer = setTimeout(() => void tick(), 2000);
    };
    void tick();
    return () => { stopped = true; clearTimeout(timer); };
  }, [documentId]);
```

```

if (!conflict) return null;
const words = conflictWords(conflict);
return (
  <div role="alert" className="conflict">
    <p><strong>A push to this page was not applied.</strong> The canvas is unchanged, and you can keep editing.</p>
    <p>Commit <code>{words.commit}</code> changed <code>{words.file}</code>: {words.why}</p>
  </div>
);
}

```

Commit and file come from an engineer's push. The banner renders them inside `{...}` in JSX, which React turns into *text nodes*, never markup; the web test's fixture has a file named `<img src=x onerror=alert(1)>.tsx` to make the point. The polling is a `setTimeout` chain with a `stopped` flag, not `setInterval`: a slow answer never overlaps the next ask, and the cleanup function React calls on unmount stops the chain. A 404 stops it for good: the canvas says so itself when the document goes.

FROM `apps/worker/src/push.ts`

```

/**
 * E5.4 (F16b): what the canvas's banner shows. A refused page is recorded with its commit and file, and it
 * replaces the conflict the document had; an applied one clears it, as the document's branch is back in
 * shape. So does Ship's own page (noon-wv8.6.1): the branch now holds the codegen byte for byte. Any other skipped
 * page (another branch, no document) says nothing about the document, so it changes nothing.
 * ponytail: only the NEWEST conflict is kept; ceiling: two refused pushes in a row show only the second;
 * upgrade: a row per event, if engineers ever need the list.
 */
export async function keepConflict(store: Pick<GitStore, "recordConflict" | "clearConflict">, event: GitEvent, page: ChangedPage, outcome:
PushOutcome): Promise<void> {
  if (outcome.kind === "conflict") await store.recordConflict(page.documentId, { commit: event.after, file: page.path, reason:
outcome.reason, detail: outcome.detail });
  else if (outcome.kind === "applied" || outcome.why === "shipped") await store.clearConflict(page.documentId);
}

```

FROM `apps/api/src/conflict.int.test.ts`

```

test("a refused push is the document's conflict until one is applied; only members may read it", async () => {
  const doc = await aDocument("ann@example.com");
  const read = async () => DocumentConflict.parse(await (await as("ann@example.com", `/documents/${doc.id}/conflict`)).json());
  expect(await read()).toEqual({ conflict: null });

  const git = ctx.db.db.gitStore();
  // The file name is an engineer's: stored and answered as it was written, markup and all (the canvas renders text).
  await git.recordConflict(doc.id, { commit: "a".repeat(40), file: "src/pages/<b>old</b>.tsx", reason: "spread", detail: "line 1" });
  await git.recordConflict(doc.id, { commit, file: `src/pages/noon-${doc.id}.tsx`, reason: "hook", detail: "x".repeat(1000) });
  const { conflict } = await read();
  expect(conflict).toMatchObject({ commit, file: `src/pages/noon-${doc.id}.tsx`, reason: "hook", detail: "x".repeat(300) });

  expect((await as("bob@example.com", `/documents/${doc.id}/conflict`)).status).toBe(404);
  await git.clearConflict(doc.id);
  expect(await read()).toEqual({ conflict: null });
});

```

FROM `e2e/conflict.spec.ts`

```

/** JSON with every object's keys sorted: the same tree hashes the same whatever order its keys arrived in. */
const canonical = (value: unknown): string =>
  Array.isArray(value) ? `[${value.map(canonical).join(",")}]`
  : value !== null && typeof value === "object" ? `${Object.keys(value).sort().map((k) => `${JSON.stringify(k)}:${canonical((value as
Record<string, unknown>)[k])}`).join(",")}`
  : JSON.stringify(value);

/** What the room holds NOW, as it welcomes a new peer: the tree's hash and the seq it has reached. Opening also asks the git peer to
reconcile. */
async function roomState(browser: Browser, url: string): Promise<{ tree: string; seq: number }> {
  const { doc, seq } = await welcomeOf(browser, url);
  return { tree: createHash("sha256").update(canonical(doc)).digest("hex"), seq };
}

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>Record<ConflictReason, string> : the compiler demands every key</code>	an <code>EnumMap</code> does not; an exhaustive <code>switch</code> over a sealed type does	<code>dict[Literal[...], str]</code> is not checked for completeness; <code>assert_never</code> in a match is
<code>z.string().transform((d) => d.slice(0, 300)) : trim, do not refuse</code>	a <code>@Size</code> would refuse; a setter that truncates	a validator that returns the trimmed value
<code>on conflict (document_id) do update set x = excluded.x</code>	the same SQL; <code>MERGE</code> in newer engines	the same SQL
<code>{words.commit}</code> in JSX: a text node	Thymeleaf's <code>th:text</code> escapes; <code>th:utext</code> does not	Jinja2 autoescape on
a <code>setTimeout</code> chain with a <code>stopped</code> flag in a <code>useEffect</code> cleanup	a <code>ScheduledExecutorService</code> you shut down	an <code>asyncio.Task</code> you cancel

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A message over the room, and a column on `git_events`, were both rejected as the place for a conflict.

A room message would be a new wire type and new room state for something that is not an op, and lesson 2's room has exactly one kind of thing in its log. A column on `git_events` fails on cardinality: one event can touch several documents, and the canvas asks per document. So a table keyed by document, and a plain HTTP `GET` polled like the preview. Ponytails named: only the newest conflict is kept (upgrade: a row per event); polling (upgrade: SSE).

Accessibility is not optional here: `role="alert"` makes a screen reader announce the banner when it appears, and the ship panel below uses `aria-live="polite"` for progress because the page already has one `role="status"` (the connection). The security review's list of things never to simplify away ends with accessibility for a reason.

8 · Ship: idempotent by index, safe by a push that is never forced

Ship is F17: press it, and the document's page is a commit on its branch in Gitea with an open pull request into `main`; press it again, and the same pull request shows the new page. It is a job on a queue of its own, drained by its own process, because it holds the one credential that may *push*: no model and no Docker socket beside it.

FROM `apps/worker/src/main.ts`

```
// E5.5 (F17): Ship. Its own process, as it holds the Gitea token that may push; the AI worker never does.
function shipHandlers(): Handlers {
  return {
    ship: createShipHandler({
      sessions: sync.sessions, manifest, seed: config.sandbox.seed, stopping: stopping.signal, roleOf,
      report: (job, output) => db.jobStore().report(attemptKey(job), output),
    }),
  };
}
```

FROM `packages/db/migrations/0011_ship_jobs.sql`

```
-- E5.5 (F17): the 'ship' queue. Presses of Ship coalesce into the document's WAITING ship: it has not read the
-- document yet, so it will ship every edit made before it starts. A press while one RUNS queues the next one,
-- which reads the document afresh: pressing again after an edit always ships that edit. Two ships running for
-- one document are safe (the branch refuses a push that is not on top of it, and Gitea keeps one open pull
-- request per branch), so only the waiting one is unique, and the database decides, not a look-then-insert.
create unique index jobs_one_queued_ship_per_document on jobs (document_id) where queue = 'ship' and status = 'queued';
-- The git peer asks, for each push, "did Ship make this commit?" (jobs.output->'commit').
create index jobs_ship_commit on jobs ((output ->> 'commit')) where queue = 'ship';
```

Idempotence lives in Postgres. The *queued* ship is unique per document, so twenty presses at once make one job. Read why it is not "one unfinished ship": a press while one *runs* must queue the next, because the running one has already read the document and a press after an edit must ship that edit. Two ships running for one document are safe (below), so only the waiting one needs to be unique.

FROM `packages/db/src/index.ts`

```
startShip: async ({ documentId, createdBy, idempotencyKey }) => {
  if (!orgExists || !isId(documentId) || (createdBy !== undefined && !isId(createdBy))) return undefined;
  if (idempotencyKey === undefined) return pressShip(documentId, createdBy, pool);
  if (createdBy === undefined) throw new Error("an idempotency key needs the user it belongs to");
```

```

    const keyed = await withKey(createdBy, idempotencyKey, { queue: "ship", documentId, input: {} }, (via) => pressShip(documentId,
    createdBy, via), (started) => started?.ship.id);
    if (keyed === "key_reused") return keyed;
    if ("made" in keyed) return keyed.made;
    const ship = await one(ShipRow, "select * from jobs where org_id = $1 and id = $2 and queue = 'ship'", [orgId, keyed.replay]);
    return ship && { ship, created: undefined };
  },

```

Lesson 3's lesson, applied again: the insert with `on conflict do nothing`, then a *second statement* to read the newest ship, because a CTE would share the write's snapshot and not see the winner's row. The api answers 201 when this press made the job and 200 when it joined one, and the canvas's panel polls while it runs and links the pull request (http(s) only, parsed with the contract before it goes near an `href`).

FROM `apps/worker/src/ship.ts`

```

/**
 * The 'ship' queue's handler (F17, SPEC §8 step 9): the document's page, generated afresh from the room's
 * CONFIRMED document, becomes one commit on the document's working branch ('noon/<id>', the branch its sandbox
 * works on) in Gitea, and that branch has one open pull request into main. Shipping again adds a commit to the
 * same branch, so the same pull request shows it; a branch that already holds the page as generated gets no
 * commit at all.
 *
 * The commit is built from git objects alone (a bare repo per job, a temporary index): no checkout, nothing
 * of the customer's repo ever runs here. It goes on top of whatever the branch holds, an engineer's commits and
 * an out-of-shape page included (the conflict banner does not stop a ship; the fresh page is the fix).
 *
 * Two ships of one document may run at once (0011_ship_jobs.sql says why). Neither loses: a push that is not on
 * top of the branch is refused by git, and the loser builds again on the winner; Gitea keeps one open pull
 * request per branch (409), and the loser finds it by 'head.ref'.
 *
 * The Gitea token stays in THIS process: git gets it as a header in the environment (gitEnv), the API as a
 * header; never in a URL, an argument, a log line or the job's row.
 */

```

FROM `apps/worker/src/ship.ts`

```

try {
  await git(["init", "--quiet", "--bare"]);
  for (let attempt = 1; ; attempt += 1) {
    // "--": a URL is never an option.
    const heads = remoteHeads(await remote(["ls-remote", "--", seed.url, "refs/heads/main", branch]));
    const onBranch = heads.has(branch);
    if (!onBranch && !heads.has("refs/heads/main")) throw new JobFailure("no_main_branch");
    await remote(["fetch", "--quiet", "--no-tags", "--", seed.url, `+${onBranch} ? branch : "refs/heads/main":refs/ship/parent`]);
    const parent = await git(["rev-parse", "--verify", "refs/ship/parent^{commit}"]);
    // The parent's tree with the one file the document owns (keystone 8) replaced: nothing else changes.
    await git(["read-tree", parent]);
    const blob = await git(["hash-object", "-w", "--stdin", tsx]);
    await git(["update-index", "--add", "--cacheinfo", `100644,${blob},${pagePath(documentId)}`]);
    const tree = await git(["write-tree"]);
    if (onBranch && tree === (await git(["rev-parse", `${parent}^{tree}`]))) return null;
    const commit = await git(["commit-tree", tree, "-p", parent, "-m", `Ship the page of document ${documentId}\n\nGenerated by Noon
from the document; the file is its codegen, byte for byte.`]);
    await recordCommit(commit);
    try {
      // Never forced: a branch that moved since the fetch refuses this, and nothing on it is lost.
      await remote(["push", "--", seed.url, `${commit}:${branch}`]);
      return commit;
    } catch (err) {
      if (!pushRejected(err)) throw err;
      if (attempt >= PUSH_ATTEMPTS) throw new JobFailure("branch_busy", err instanceof Error ? err.message : String(err));
    }
  }
}

```

This is *git plumbing*: a commit built from objects, with no working tree and nothing of the customer's repository ever run. `init --bare`, a temporary index (`GIT_INDEX_FILE`), `read-tree` the parent's tree into it, `hash-object -w --stdin` to store the page as a blob, `update-index --cacheinfo` to put that blob at the page's path, `write-tree`, and `commit-tree` with the parent. If the new tree equals the parent's tree, the branch already holds the page: no commit, and `null` is the answer. The parent is the branch itself, or `main` for a first ship, so an engineer's commits and an out-of-shape page stay *under* the new commit, never overwritten.

The push is never forced. A branch that moved since the fetch refuses a push that is not on top of it (git's own fast-forward rule), and the loser builds again on the winner: three attempts, then `branch_busy` as a named failure. And `recordCommit` is called *before* the push (section 6's case study): by the time the webhook sees Ship's push, the row already says the commit is Ship's.

FROM apps/worker/src/ship.ts

```
/**
 * Per document, the pull request call in flight in this process. Gitea's "one open pull request per branch" is a
 * look-then-insert on its side, not a constraint: two creates at the same instant could both pass it. So the
 * ships of one document ask one at a time. ponytail: one ship worker (compose runs one); ceiling: a second
 * process could still race Gitea; upgrade: a Postgres advisory lock per document around this call.
 */
const pullsInFlight = new Map<string, Promise<unknown>>();
function oneAtATime<T>(key: string, work: () => Promise<T>): Promise<T> {
  const next = (pullsInFlight.get(key) ?? Promise.resolve()).catch(() => undefined).then(work);
  pullsInFlight.set(key, next);
  const forget = (): void => { if (pullsInFlight.get(key) === next) pullsInFlight.delete(key); };
  next.then(forget, forget);
  return next;
}
```

FROM apps/worker/src/ship.ts

```
const opened = await call("POST", "/pulls", { head: branch, base: "main", title: `Noon: the page of document ${documentId}`, body:
`Generated by Noon. \`${pagePath(documentId)}\` is the document's codegen, byte for byte; each ship adds a commit here.` });
if (opened.status === 201) return pullOf(Pull.parse(await opened.json()));
if (opened.status !== 409) throw new JobFailure("gitea_unavailable", `POST /pulls -> ${String(opened.status)}`);
for (let page = 1; page <= MAX_PULL_PAGES; page += 1) {
  const listed = await call("GET", `/pulls?state=open&limit=${String(PULLS_PER_PAGE)}&page=${String(page)}`);
  if (!listed.ok) throw new JobFailure("gitea_unavailable", `GET /pulls -> ${String(listed.status)}`);
  const items: unknown = await listed.json();
  const found = openPullOf(items, branch);
  if (found) return found;
  if (!Array.isArray(items) || items.length < PULLS_PER_PAGE) break;
}
// ponytail: more than MAX_PULL_PAGES pages of open pull requests is not searched; upgrade: Gitea's head filter.
throw new JobFailure("gitea_unavailable", "Gitea says the branch has a pull request, and none is open");
}
```

Gitea's "one open pull request per branch" is a look-then-insert on *its* side, so two creates in the same instant could both pass it; the ships of one document ask one at a time in this process (the same promise-chain mutex as section 1's mirrors), with the ceiling named (a second ship worker) and the upgrade (an advisory lock). A 409 means the branch has its pull request, which is found by `head.ref` across pages of the listing. And the fetch that carries the token has `redirect: "error"`: a token for Gitea never follows a redirect to anywhere else.

FROM apps/worker/src/ship.ts

```
try {
  return await fetchImpl(`${api}${path}`, {
    method,
    // `redirect: "error"`: the token is for Gitea, and never follows a redirect anywhere else.
    redirect: "error",
    signal: AbortSignal.any([signal, AbortSignal.timeout(timeoutMs)]),
    headers: { "content-type": "application/json", ...(seed.auth ? { authorization: `token ${seed.auth.token}` } : {}) },
    ...(body === undefined ? {} : { body: JSON.stringify(body) }),
  });
} catch (err) {
  if (signal.aborted) throw err;
  throw new JobFailure("gitea_unavailable", `${method} ${path}: ${err instanceof Error ? err.message : String(err)}`);
}
};
```

FROM apps/worker/src/ship.ts

```
return async (job, cancelled) => {
  const userId = job.createdBy;
  if (userId === undefined) throw new JobFailure("owner_missing");
  requireEditor(await roleOf(job.documentId, userId)); // POST /ship needs an editor; a viewer by now may not push (noon-87s)
  const signal = AbortSignal.any([cancelled, stopping]);
  try {
    const generated = generate(await readDocument(job, userId), manifest);
    if (!generated.ok) throw new JobFailure("codegen_failed", generated.reason);
    const commit = await shipPage({ seed, documentId: job.documentId, tsx: generated.tsx, signal, recordCommit: (made) => report(job, {
      commit: made, pr: null }) });
    const pr = await oneAtATime(job.documentId, () => ensurePull({ seed, documentId: job.documentId, signal, ...(fetchImpl ? { fetchImpl
    } : {}) }));
    await report(job, { commit, pr });
    return undefined;
  } catch (err) {
    // A stopping worker names why, as an AI run does; a cancel ends as cancelled (the worker says so).
    if (stopping.aborted) throw new JobFailure("worker_stopped");
    throw err;
  }
}
```

```
};
}
```

FROM apps/web/src/ShipPanel.tsx

```
const sentence = (ship: Ship): string =>
  ship.status === "queued" ? "Waiting to ship."
  : ship.status === "running" ? "Shipping: pushing the page and opening the pull request."
  : ship.status === "succeeded" ? (ship.commit === null ? "Shipped. The pull request already had this page." : "Shipped. The pull request
has the page as it is now.")
  : ship.status === "cancelled" ? "The ship was cancelled."
  : `Shipping failed. ${FAILURES[ship.error ?? ""]} ?? "Something went wrong on the server."`;
const active = (ship: Ship | null): ship is Ship => ship?.status === "queued" || ship?.status === "running";
```

FROM apps/worker/src/ship.int.test.ts

```
test("Ship once, Ship again: ONE open pull request, whose file is byte-identical to a fresh codegen of the document each time", async ()
=> {
  await edit(button("b1", "Pay"));
  const first = await ship();
  const branch = `noon/${documentId}`;
  expect(first?.commit).toMatch(/^[0-9a-f]{40}$/u);
  expect(first?.pr).toMatchObject({ number: expect.any(Number) as number, url: expect.stringContaining(`/noon/${REP0}/pulls/`) as string
});
  expect(await pageOnBranch(documentId)).toBe(fileOf(doc));
  expect(await openPulls(branch)).toEqual([expect.objectContaining({ number: first?.pr?.number, head: expect.objectContaining({ sha:
first?.commit }) as unknown })]);

  await edit(button("b2", "Cancel"));
  await edit({ type: "set_prop", nodeId: "b1", key: "label", value: "Pay now" });
  const second = await ship();
  expect(second?.pr).toEqual(first?.pr); // the same pull request, updated
  expect(second?.commit).not.toBe(first?.commit);
  expect(await pageOnBranch(documentId)).toBe(fileOf(doc));
  expect(await openPulls(branch)).toEqual([expect.objectContaining({ number: first?.pr?.number, head: expect.objectContaining({ sha:
second?.commit }) as unknown })]);

  // Nothing changed since: no commit, the branch where it was, the same pull request.
  const third = await ship();
  expect(third).toEqual({ commit: null, pr: first?.pr });
  expect(await tipOf(documentId)).toBe(second?.commit);
  // The commit was reported BEFORE it was pushed: the git peer must know a push is Ship's by the time it sees it.
  expect(outputs.slice(0, 2).map((each) => each.output)).toEqual([
    { commit: first?.commit, pr: null },
    { commit: null, pr: first?.pr }
  ]);
});
```

FROM apps/api/src/ship.int.test.ts

```
test("twenty presses at once make ONE ship, enqueued once; every press is answered with it", async () => {
  const doc = await shippable("ann@example.com");
  expect(await newest("ann@example.com", doc)).toBeNull();
  const answers = await Promise.all(Array.from({ length: 20 }, () => ship("ann@example.com", doc)));
  expect(answers.map((a) => a.status).sort()).toEqual([...Array<number>(19).fill(200), 201]);
  const ships = await Promise.all(answers.map(async (a) => Ship.parse(await a.json())));
  const [job] = await shipJobs(doc);
  expect(await shipJobs(doc)).toEqual([
    { id: job?.id, status: "queued" }
  ]);
  expect(new Set(ships.map((s) => s.id))).toEqual(new Set([job?.id]));
  expect(enqueued.filter((ref) => ref.jobId === job?.id)).toEqual([
    { queue: "ship", jobId: job?.id, orgId: doc.orgId }
  ]);
});

test("a press while a ship RUNS queues the next one; the canvas reads the newest, with its commit and pull request", async () => {
  const doc = await shippable("ann@example.com");
  const first = Ship.parse(await (await ship("ann@example.com", doc)).json());
  const key = { queue: "ship" as const, jobId: first.id, orgId: doc.orgId };
  await ctx.db.db.jobStore().claim(key);
  const pr = { number: 4, url: "http://localhost:3002/noon/sample-app/pulls/4" };
  await ctx.db.db.jobStore().report(key, { commit: "c".repeat(40), pr });
  expect(await newest("ann@example.com", doc)).toMatchObject({ id: first.id, status: "running", commit: "c".repeat(40), pr });
  expect(await ctx.db.db.gitStore().shippedCommit("c".repeat(40))).toBe(true); // what the git peer skips
  expect(await ctx.db.db.gitStore().shippedCommit("d".repeat(40))).toBe(false);

  const second = await ship("ann@example.com", doc);
  expect(second.status).toBe(201);
  expect(Ship.parse(await second.json())).toMatchObject({ status: "queued", commit: null, pr: null });
  expect((await shipJobs(doc)).map((j) => j.status)).toEqual(["running", "queued"]);
});
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>git hash-object , update-index --cacheinfo , write-tree , commit-tree</code>	<code>JGit: ObjectInserter , DirCache , CommitBuilder</code>	<code>dulwich: Blob , Tree , Commit objects</code>
<code>AbortSignal.any([cancelled, stopping, AbortSignal.timeout(ms)])</code>	<code>a CompletableFuture composed with orTimeout and a cancel flag</code>	<code>asyncio.wait_for</code> plus a cancellation scope
<code>fetch(url, { redirect: "error" })</code>	<code>HttpClient.Redirect.NEVER</code>	<code>requests.get(allow_redirects=False)</code> and check
<code>z.url({ protocol: /^https?\$/u })</code> before an href	<code>URI.create</code> then a scheme check	<code>urllib.parse</code> then a scheme check
a partial unique index on <code>status = 'queued'</code>	the same SQL; JPA cannot express it	the same; Django <code>UniqueConstraint(condition=...)</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

"One unfinished ship per document" would have folded a press during a run into a ship that had already read the older document.

The obvious index, mirroring the sandbox job's, is wrong here because of *when the document is read*. A sandbox job follows the document for as long as it runs; a ship reads it once, at the start. So the second press during a run must become a second job, and only the waiting one is unique. The test presses twenty times at once (one job), then presses while one runs (a second job), and the canvas reads the newest.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Two ships at once: neither loses, and there is still one pull request.

Both build a commit on the branch as they fetched it. One push wins; the other is refused as not a fast-forward and builds again on top, and the branch ends holding the page. Both then ask for a pull request; the second gets a 409 and finds the first's by `head.ref`. Note what makes this safe without a lock: git's own refusal rule and Gitea's own uniqueness, both on *their* side, and code that treats each refusal as a normal answer with a name.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Ship ships *on top* of an engineer's out-of-shape page, never over it.

The conflict banner is up: an engineer pushed a page with a hook in it. Ship's commit has that commit as its parent, and its tree is the parent's tree with the one file replaced; the branch is back in shape and the engineer's history is intact. The `e2e ( e2e/ship.spec.ts )` does exactly this with the banner up, twice, and checks the page in Gitea against a fresh generation of the final document, byte for byte. Byte equality is right here because the file is *ours*: that is the other half of section 3's argument.

Principal-level point: idempotence is a property of a *key*, and the key must be chosen from what the operation reads and when. Ship reads at start, so the key is "a ship that has not started yet". A preview reads continuously, so its key is "a preview that has not ended". Same table, same `on conflict do nothing`, different predicate, for a reason you can state.

9 · Testing a loop that spans four processes and a git server

The loop crosses the canvas, the api, Postgres, the sync server, two worker processes, Docker and Gitea. The tests get three things right that no unit could.

Git fetches a path as it fetches a URL, so a local bare repository stands in for Gitea. The git peer's and Ship's integration tests run against `localOrigin()`: a bare repo and a clone in a temp directory, where "an engineer" commits and pushes. No Gitea, no network, and the code under test is the real `createGitPeer`, the real `pushOps`, the real `shipPage`. Two suites do need the dev stack's Gitea (`gitea.int.test.ts` for the bootstrap, `ship.int.test.ts` for pull requests) and say so in their names.

FROM `apps/worker/src/git-testing.ts`

```
// Shared by the integration tests of the git peer and Ship: a local bare repo stands in for Gitea (git fetches and
// pushes a path as it does a URL), and a clone of it is where "an engineer" commits and pushes.
export const exec = promisify(execFile);
const who = { GIT_AUTHOR_NAME: "eng", GIT_AUTHOR_EMAIL: "eng@localhost", GIT_COMMITTER_NAME: "eng", GIT_COMMITTER_EMAIL: "eng@localhost"
```

```
};
export const git = async (cwd: string, ...args: string[]): Promise<string> => (await exec("git", args, { cwd, env: { ...process.env, ...who } })).stdout.trim();
```

The e2e layer runs the real workers from source and stops the compose ones. Playwright starts a sandbox worker, a git peer and a ship worker as its own servers, each printing a line when ready; and because they share Postgres and Redis with the dev stack, the compose workers would take e2e's jobs, so `setup.ts` stops all four for the duration and `teardown.ts` starts them again.

FROM `e2e/setup.ts`

```
// The e2e layer shares Postgres and Redis with the dev stack, so the dev stack's WORKERS would take
// the jobs these tests create (their sweeps read the same `jobs` table): `worker` would hand runs to the
// real model, and `worker-sandbox` would start e2e's previews in the compose pool instead of the e2e
// worker's own; `worker-git` would take e2e's pushes and edit through the compose sync; `worker-ship` would ship
// e2e's documents reading the compose sync. All four are stopped for the duration; teardown.ts starts them again.
// If Playwright is KILLED (kill -9), teardown never runs: `docker compose start worker worker-sandbox worker-git worker-ship`.
// ponytail: a database and a Redis of its own for e2e is the upgrade.
```

FROM `playwright.config.ts`

```
// The REAL git peer (E5.3b), from source, against the dev stack's Gitea, with a mirror of its own. setup.ts
// stops the compose one: both would drain the same inbox, and that one would edit through the compose sync.
{ command: fromEnv(`WORKER_QUEUE=git SEED_REPO=http://127.0.0.1:${GITEA_PORT:-3002}/noon/sample-app.git
GIT_PEER_DIR=${join(tmpdir(), `${SANDBOX_POOL}-git`)}` SYNC_URL=${syncNodes} node apps/worker/src/main.ts`), wait: { stdout: /git peer
watching/u }, reuseExistingServer: false },
// The REAL ship worker (E5.5), from source, against the dev stack's Gitea; setup.ts stops the compose one.
{ command: fromEnv(`WORKER_QUEUE=ship SEED_REPO=http://127.0.0.1:${GITEA_PORT:-3002}/noon/sample-app.git SYNC_URL=${syncNodes} node
apps/worker/src/main.ts`), wait: { stdout: /worker draining queues: ship/u }, reuseExistingServer: false },
```

Prove "nothing changed" with a hash, and "as git" with the frames. The conflict e2e asks the room for its document as a fresh joiner is welcomed with it, hashes it as canonical JSON (keys sorted: key order is not part of a page), and compares before and after the push, `seq` included. The push e2e records every WebSocket frame the canvas receives and asserts the op arrived stamped `actor.kind = "git"` with the commit as `runId`; and it plants a value on `window` before the push and reads it after, because a reload would lose it.

FROM `e2e/gitea.ts`

```
// Both sides of an F16 test: the canvas's document, and an engineer's push to the dev stack's REAL Gitea (one
// commit on the document's branch, `noon<id>`, that writes its generated page). Only the one key this needs
// leaves .env, and it goes to git as a header, never into a URL or a log.
const exec = promisify(execFile);
const TOKEN = parseEnv(readFileSync(".env", "utf8"))["GITEA_TOKEN"] ?? "";
const REPO = `http://127.0.0.1:${process.env["GITEA_PORT"]} ?? "3002"/noon/sample-app.git`;
const gitEnv = {
  ...process.env, GIT_TERMINAL_PROMPT: "0", GIT_AUTHOR_NAME: "eng", GIT_AUTHOR_EMAIL: "eng@localhost", GIT_COMMITTER_NAME: "eng",
  GIT_COMMITTER_EMAIL: "eng@localhost",
  GIT_CONFIG_COUNT: "1", GIT_CONFIG_KEY_0: "http.extraHeader", GIT_CONFIG_VALUE_0: `Authorization: Basic
${Buffer.from(`noon:${TOKEN}`).toString("base64")}`,
};
```

FROM `e2e/push.spec.ts`

```
test("a push that edits the page's generated file changes the open canvas, as git, without a reload", async ({ page, browser }) => {
  // Every frame the canvas receives: the ops must arrive stamped by the room as the git peer's.
  const { documentId, buttonId, frames } = await documentWithButton(page, user);
  await page.evaluate(() => { (window as unknown as { noonMark: string }).noonMark = "same load"; }); // a reload would lose it

  // Pushed to Gitea on the document's branch.
  const pushed = await pushPage(documentId, relabelled(buttonId, "From git"), "relabel the button");
  try {
    // A new branch is found by the reconcile (the webhook ignores it): opening the document asks for one now.
    const other = await (await browser.newContext()).newPage();
    await other.goto(page.url());
    await expect(other.getByRole("status")).toHaveText("live");

    await expect(page.locator("[data-component=Button] > .node-props")).toHaveText("label=From git", { timeout: 20_000 });
    expect(await page.evaluate(() => (window as unknown as { noonMark?: string }).noonMark)).toBe("same load");
    const ops = frames.map((f) => JSON.parse(f) as { type: string; actor?: { kind: string; runId?: string }; op?: unknown });
    expect(ops.filter((m) => m.type === "op" && m.actor?.kind === "git")).toEqual([
      expect.objectContaining({ actor: expect.objectContaining({ kind: "git", runId: pushed.commit }), op: { type: "set_prop", nodeId:
buttonId, key: "label", value: "From git" } }]),
    ]);
  } finally {
    await pushed.remove();
  }
});
```

FROM e2e/ship.spec.ts

```
// An edit, and Ship again: the same pull request now holds the new page.
await page.getByRole("option", { name: "Button", exact: true }).click();
await expect(page.locator("[data-component=Button]")).toHaveCount(2);
await expect(page.getByText("saved", { exact: true })).toBeVisible();
const final = pageOf((await welcomeOf(browser, page.url()).doc);
await page.getByRole("button", { name: "Ship", exact: true }).click();
await expect.poll(() => pageInGitea(documentId), { timeout: 30_000 }).toBe(final);
await expect(status).toHaveAttribute("data-ship-status", "succeeded", { timeout: 30_000 });
const second = await openPullsOf(documentId);
expect(second.map((pull) => pull.number)).toEqual(first.map((pull) => pull.number));
expect(second[0]?.head.sha).not.toBe(first[0]?.head.sha);

// Ship's own push is not read back as an engineer's edit: the canvas still has exactly its two buttons, unlabelled by git.
await expect(page.locator("[data-component=Button]")).toHaveCount(2);
await expect(page.locator("[data-component=Button] > .node-props").first()).toHaveText("label=Button");
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
a bare repo in <code>tmpdir()</code> as the remote	JGit's in-memory repository; or the same bare repo	the same
Playwright <code>webServer</code> entries with a <code>wait: { stdout }</code>	Testcontainers' <code>LogMessageWaitStrategy</code>	<code>wait_for_logs</code>
<code>page.on("websocket", ...)</code> to read the frames the canvas received	no equivalent in Selenium; a proxy	Playwright for Python: the same API
a value planted on <code>window</code> to prove there was no reload	the same trick in any browser driver	the same
canonical JSON (sorted keys) hashed with sha256	Jackson's <code>ORDER_MAP_ENTRIES_BY_KEYS</code> , then a digest	<code>json.dumps(sort_keys=True)</code> , then <code>hashlib</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Two integration files held the same forty lines, and the duplicate detector said so.

`make check` runs `jscpd`. E5.3b's push tests and E5.3a's git peer tests each grew a local-origin helper, and the gate refused the copy. The helper moved to `git-testing.ts`, and in the same pass the room-joining and wait-for-live code shared by the preview, the push applier and Ship moved to `live.ts`. A duplicate detector in the gate is how a codebase written bead by bead, by different sessions, stays one codebase.

Honest status, repeated here because it matters: every integration and e2e test named in this chapter is written, reviewed and committed, and passed in the clean-clone gate at `6700e74` on 2026-10-02. The unit tests (parse, webhook, push-ops, ship's pure helpers, the git peer's parsers, the conflict sentences) run green in the gate on every commit. The section below is the same loop by hand.

10 · The full loop, by hand

The loop runs end to end in the gate: the `push`, `conflict` and `ship` e2e specs drive it through the real stack. These are the same steps by hand, the exact commands, in order, and what each one should show. Where a command needs the token, it is read from `.env` into the shell and never printed.

0 · BRING THE STACK UP AND WATCH THE TWO NEW WORKERS

```
./init.sh # Postgres, Redis, Gitea (127.0.0.1:3002), api, sync, worker, worker-sandbox, worker-git,
worker-ship
# the seed pushed as ONE commit; the push webhook registered; PASS: ... seed in Gitea, hook

installed
docker compose ps # gitea, worker-git and worker-ship are listed and healthy
docker compose logs -f worker-git worker-ship
# git peer watching http://gitea:3000/noon/sample-app.git (token present)
# worker draining queues: ship (to http://gitea:3000/noon/sample-app.git, token present)
```

1 · SHIP → PULL REQUEST

```
open http://localhost:5173/?user=you@example.com # New document, Add Button; note ?doc=<id> in the address bar
# Press Ship. Within seconds the panel says "Shipped. The pull request has the page as it is now." and links "Pull request #1".
GITEA_TOKEN=$(grep '^GITEA_TOKEN=' .env | cut -d= -f2-) # into this shell only; never echo it
curl -s -H "Authorization: token $GITEA_TOKEN" 'http://localhost:3002/api/v1/repos/noon/sample-app/pulls?state=open' \
  | jq '.[0] | {number, branch: .head.ref}' # exactly one, whose branch is noon/<id>
curl -s -H "Authorization: token $GITEA_TOKEN" "http://localhost:3002/api/v1/repos/noon/sample-app/raw/src/pages/noon-<id>.tsx?"
```

```
ref=noon%2F<id>"

# the page: one import, export function Page, label={"Button"}
```

2 · PUSH → CANVAS, AS AN ENGINEER

```
AUTH="http.extraHeader=Authorization: Basic $(printf 'noon:%s' "$GITEA_TOKEN" | base64)" # the token as a header, never in the URL
git -c "$AUTH" clone http://localhost:3002/noon/sample-app.git /tmp/sample-app && cd /tmp/sample-app
git checkout noon/<id>
sed -i 's/label={"Button"}/label={"From git"}/' src/pages/noon-<id>.tsx
git commit -am 'relabel the button' && git -c "$AUTH" push origin HEAD
# Back in the canvas, within about two seconds and without a reload: the button reads "From git".
# Both doors, in Postgres: the webhook's row has a delivery id; a row the reconcile found does not.
docker compose exec -T postgres psql -U noon -d noon -c \
    "select ref, left(after_sha, 8) as commit, status, delivery_id is not null as via_webhook from git_events order by created_at"
```

3 · BREAK THE SHAPE → BANNER, NOTHING ELSE

```
# In the same clone: add a hook as the first line of Page(), then push.
sed -i 's/^export function Page() {$/export function Page() {\
    const [count] = useState(0);/' src/pages/noon-<id>.tsx
git commit -am 'count clicks' && git -c "$AUTH" push origin HEAD
# The canvas shows the banner: "A push to this page was not applied." with the commit, the file and "It uses a React hook."
# The button still reads "From git"; adding a Button still works, with the banner up.
curl -s -H 'x-dev-user: you@example.com' http://localhost:3000/documents/<id>/conflict | jq # commit, file, reason: hook
```

4 · SHIP AGAIN → THE SAME PULL REQUEST, BACK IN SHAPE

```
# Press Ship. The same "Pull request #1" gains a commit whose parent is the engineer's; the page is in shape again.
curl -s -H "Authorization: token $GITEA_TOKEN" 'http://localhost:3002/api/v1/repos/noon/sample-app/pulls?state=open' | jq 'length' # 1
docker compose logs worker-git | grep skipped # Ship's own push: {"kind":"skipped","why":"shipped"}; the canvas is untouched by it
```

5 · THEN THE SUITES THAT PROVE IT WITHOUT HANDS

```
pnpm exec vitest run --config vitest.integration.config.ts apps/api/src/webhook.int.test.ts apps/worker/src/git.int.test.ts
apps/worker/src/push.int.test.ts
# local bare repos stand in for Gitea; Postgres and the sync server are real
pnpm exec vitest run --config vitest.integration.config.ts apps/worker/src/gitea.int.test.ts apps/worker/src/ship.int.test.ts
# the dev stack's Gitea: the bootstrap, and ship-once-twice-same-pr-byte-identical
pnpm exec playwright test e2e/push.spec.ts e2e/conflict.spec.ts e2e/ship.spec.ts
# the real canvas; stops the compose workers for the duration (see e2e/setup.ts)
make check # the whole gate, as the pre-commit hook runs it
```

What to do when a step does not match this page: the page was cut from the code at commit `d4eea6f`, and the code is right until a test says otherwise. Read the log line, find the section above that owns that line, and file the difference against the bead that owns the code. A chapter that disagrees with a running system is a bug report, whichever of them is wrong.

11 · What the type system could not see, epic 5 edition

Everything below compiled under `strict`, passed the linter and the first tests, or was rejected at design time by asking what the types could not ask.

WHAT WENT WRONG, OR WOULD HAVE	WHY	FIX
a sandbox on an internal network could not clone	Docker publishes no port for an internal-only container (measured)	the worker fetches; a bundle by <code>docker exec</code>
ICC-off on one shared network cut the proxy off too	measured, not read	one network per document, the proxy joined to each
a document id alone opened any preview through the public URL	the URL was the only lock	a per-container token: nonce + HMAC, checked twice
Vite's 404 named the base	read in Vite's source, measured on a local run	a <code>pre</code> plugin answers a bare 404 first
a blank frame through the tunnel	Vite answers preflights before its proxy; origin "null" refused	<code>preflightContinue</code> for <code>/preview/</code> only
401s on the preview's modules	the tunnel's basic auth; the frame carries no credentials	fixed in the tunnel; know which hop answered
a quoted attribute with <code>&</code> would round-trip to another value	JSX decodes entities; the parser does not	refuse, and say how to write it
deep nesting overflowed TypeScript's parser	its walk recurses; ours does not	<code>RangeError</code> is <code>too_deep</code>
<code>parseDiagnostics</code> is internal to TypeScript	the public route is a whole Program	cast through <code>unknown</code> ; fail closed if it is gone
a replayed body under a fresh delivery id was a new event	the header is not signed	a second unique key on (branch, commit)
a push during a reconcile fetch could be lost	the flag was taken after the fetch	take the flag first
an event failed when Gitea was away	"failed" is terminal and the reconcile never re-records	back to <code>pending</code>
a symlink named like a page read the worker's environment	<code>readFile</code> follows links	<code>O_NOFOLLOW</code> , <code>isFile()</code> , size first
a two-way diff reverted every canvas edit since the file was generated	the wrong base	three-way: base, target, current
a removed id came back as a new node	an id is a name, not a value	<code>earlierIds</code> from <code>git log -p</code> ; refuse whole
Ship's push came back as an engineer's edit and undid a racing canvas edit	the webhook cannot tell who pushed	record the commit BEFORE the push; skip it
a force-push $A \rightarrow B \rightarrow A$ reads as known	keyed on (branch, commit)	named ponytail; key on before as well
two ships in one instant could both create a pull request	Gitea's uniqueness is a look-then-insert	one at a time per document; $409 \rightarrow$ find by <code>head.ref</code>
a press during a run folded into a ship that read the older document	"unfinished" was the wrong predicate	unique on <code>status = 'queued'</code>
a CTE would not see the winner's row	one snapshot per statement	a second statement, again
a token could follow a redirect off Gitea	fetch follows redirects by default	<code>redirect: "error"</code>
forty duplicated lines across two suites	written bead by bead	<code>jscpd</code> in the gate; <code>git-testing.ts</code> , <code>live.ts</code>
the per-org cap could overshoot	a count is not an index (lesson 4)	an advisory lock per org; the named upgrade, taken

The pattern, extended once more: lessons 2 to 4 said types prove shapes and say nothing about order, time, repetition, memory, who is speaking, what a subprocess does, or a browser boundary. Epic 5 adds: types cannot see *which delivery is the same event, which starting point a diff should use, or which side of a network a container is on*. Every one was answered the same way as before: a key chosen from what the operation reads, a measurement of the real thing, and a name for the failure.

Your turn: three drills

Do these alone. They live in `drills/lesson-5/` and both coded drills start RED; `sh drills/lesson-5/check.sh` shows your progress.

D1 · Extend a type, end to end. The banner says what was pushed and why it was refused, and not by whom. Gitea's payload carries `pusher.login`. Carry it from the webhook to the banner: the reading (`PushReading`'s event gains `pusher`, validated, dropped rather than fatal when strange), a migration (`pusher` on `git_events` and `document_conflicts`, with a check), the store (`GitEvent`, `record`, `recordConflict`, `ConflictRow`), the `Conflict` contract (a `strictObject`: the api cannot even parse the row until you do), the route, `keepConflict`, and `conflictWords` (`who`). make `check` must stay green: the type checker will name every place a `Conflict` is built without a `pusher`, and that list is the drill.

Predict first: a reconcile finds a push with no delivery. What does the banner say about who pushed, and why must that not be an error?

D2 · Fix a planted bug. `drills/lesson-5/d2-three-way/push-ops.ts` is "a push becomes ops" cut down to adds, props and removes, with one bug from this chapter planted in it. Its test builds the section 6 case: canvas edits since the file was generated, and an engineer's push from the old file. Read the failing assertion and say, before you open the code, whose work got undone. The fix changes one line.

D3 · Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. The webhook already dedupes on X-Gitea-Delivery. Why is there a second unique key on (`ref`, `after_sha`), and why does dedupe live in Postgres rather than in the api?

Two reasons for the second key. First, the delivery id is a header and the HMAC signs only the body, so anyone who captured one delivery can replay the body under a made-up id and pass every check; a dedupe on the id alone is not a dedupe. Second, Gitea never retries a failed delivery, so the git peer reconciles its mirror on a timer and when a document is opened, and a reconcile has no delivery id at all. The reconcile and the webhook must meet somewhere, and (`branch`, `commit`) is the content they both know. It lives in Postgres because two doors can arrive in the same instant: a "select, then insert" in the api would let both pass, while `insert ... on conflict do nothing returning id` is one statement that Postgres serialises, and `rowCount` tells the caller who won. The test sends ten copies at once and expects one 202. The named ceiling: a branch force-pushed `A → B → A` reads as known; the upgrade is a key on (`ref`, `before`, `after`).

▼ 2. The git peer could write ops into Postgres directly; it is a trusted process on the compose network. Why does it instead open a WebSocket to our own sync server and submit ops like a browser tab? And why is the diff three-way?

Because keystone 2 says one write path, and everything epic 2 built hangs off it: the room orders ops and assigns `seq`, checks each op against the tree as it is *now*, charges the actor's budget, relays to the other peers, and (epic 6) journals for recovery. A second writer would have to reimplement all of that or bypass it, and either way two tabs and a push would no longer converge by one rule. Going through `peer-client` also means the room can refuse the git peer's ops one by one, exactly as it refuses a tab's, and every op is stamped `actor.kind = "git"` with the commit as `runId`, which is what the canvas shows and the e2e asserts. The diff is three-way because the engineer started from a file generated at some past moment, and the canvas has edited since. Diffing the pushed file against the document treats every one of those canvas edits as something the engineer reverted (drill 2 makes you watch it happen). Diffing base against target, and replaying onto the current document with each op tried on a copy first, sends only what the engineer changed, and lets a remove on the canvas beat an edit in the push, the same rule the room applies to two tabs.

▼ 3. "One unfinished sandbox job per document" was the right index in epic 4. Ship's index is "one *queued* ship per document". Why the different predicate, and why does Ship record its commit on the job *before* pushing it?

Because idempotence is a property of a key, and the key must be chosen from what the operation reads and when. A sandbox job reads the document continuously for as long as it runs, so a second one would be a duplicate at any point in its life: "unfinished" is right. A ship reads the document once, at the start, and then pushes what it read. A press while one runs must therefore become a *second* job, or the edit that prompted the press would be folded into a ship that already read the older document and would never ship. Only the waiting one is unique, so twenty presses at once make one job and a press during a run makes the next. Two ships running at once are safe without a lock because git refuses a push that is not a fast-forward (the loser builds again on top, three times, then `branch_busy`) and Gitea keeps one open pull request per branch (409, and the loser finds it by `head.ref`). The commit is recorded before the push because Ship's push comes back through the webhook like anyone's: the git peer asks "did Ship make this commit?" before it reads or opens anything, and if the answer were written after the push, the webhook could arrive first, diff Ship's page against the room, and undo a canvas edit made while the ship ran.

Recall first

Look away. Name the two doors a push can come through and the one place they meet. Then name the three documents in the three-way diff and what each one is. Then check.

▼ Check yourself

Two doors: the api's `POST /webhooks/gitea` (HMAC over the raw bytes, then one row with the delivery id), and the git peer's reconcile of its bare mirror (on a 30 s timer and when a document is opened; a row with no delivery id). They meet at `git_events`'s unique key on `(ref, after_sha)`, and `on conflict do nothing` makes the second arrival a duplicate whichever it was. Three documents: *base*, the page at the commit before the push (what the engineer started from); *target*, the pushed page; *current*, the room's confirmed document as it is now. The ops are the base-to-target changes, each tried on a copy of current and sent only if it changes something.

Then test it

1. Gitea signs the webhook body. What exactly does the api compute the HMAC over, and why?

The body plus the X-Gitea-Delivery header

The parsed JSON, re-serialised with sorted keys

The raw request bytes, before any parsing: a parse-and-reserialise would verify something Gitea never sent, and one byte of whitespace is another body

The ref, before and after fields only

The header is not covered by the signature at all, which is why a replayed body under a fresh delivery id needs the second unique key.

2. Before comparing the signature, the api checks the header against `/^[0-9a-f]{64}$/`. Why is the regex there?

`timingSafeEqual` throws on buffers of different lengths, so the length is fixed first; and a malformed header is refused before any compare

To make the compare faster

To reject the sha256= prefix GitHub uses

Because `Buffer.from(hex)` throws on odd lengths

The same two lines guard the sandbox proxy's token: a regex fixes the length, then a constant-time compare.

3. Gitea never retries a failed webhook delivery. What is the second door, and when does it open?

The git peer fetches its bare mirror and records every branch whose commit is not the newest one recorded: every 30 s, and when a document is opened (a one-row flag the peer takes BEFORE fetching)

The api asks Gitea to redeliver

A cron job replays the webhook log

The canvas polls Gitea's API for new commits

Taking the flag before the fetch means an open that arrives during the fetch sets it again and is not lost. The two doors meet at `(ref, after_sha)`.

4. A commit contains a symlink at `src/pages/noon-<id>.tsx` pointing at `/proc/self/environ`. What does the git peer do?

Reads it: the worktree is a checkout of trusted repository content

Refuses every commit containing any symlink

Opens the page with `O_NOFOLLOW`, gets `ELOOP`, and refuses it as `not_a_file`: a conflict the banner shows, and nothing outside the worktree is read

Follows the link but only inside the container

The worker's environment holds the Gitea token. `readFile` follows links; `open(O_NOFOLLOW)` does not, and `stat().isFile()` refuses a directory (a submodule).

5. The canvas renamed node A after the file was generated; the engineer's push, from the old file, relabels B. What does a two-way diff (pushed file against the document) send, and what does the three-way diff send?

Both refuse the push as a conflict

Two-way: relabel B AND rename A back, undoing the canvas. Three-way (base → target, replayed on current): relabel B only

Both send relabel B only; the difference is performance

Two-way sends nothing; three-way sends both

Drill 2 plants the two-way version. The fix is one expression: what `from` is.

6. Why does Ship record its commit on the job row BEFORE pushing it to Gitea?

To let a second ship reuse the commit

Ship's push comes back through the webhook like anyone's; the git peer skips a commit Ship made, and if the record came after the push the webhook could win the race and diff Ship's page against the room, undoing a canvas edit made during the ship

Because Postgres requires the output before the job can finish

So the canvas can show the commit hash while the push is in flight

The order is the whole fix. The integration test asserts the first reported output is `{ commit, pr: null }`, before the push.

7. Ship's unique index is on `queue = 'ship' and status = 'queued'`, not on "unfinished" like the sandbox job. Why?

A ship reads the document once, at start; a press while one runs must queue a NEXT ship that reads afresh, or the edit that prompted the press would never ship. Only the waiting one coalesces presses

Postgres cannot index on two status values

To let the canvas cancel a queued ship

Because two ships running at once would corrupt the branch

Two ships at once are safe: a push is never forced (the loser builds again on top), and Gitea's 409 is answered by finding the open pull request by `head.ref`.

8. Every sandbox is on its own `--internal` Docker network. How does the canvas reach a preview, and how does the sandbox get its clone?

Docker publishes the sandbox's port anyway; the sandbox clones from Gitea over the compose network

The proxy has a route to Gitea and the sandbox clones through it

Through the pool's proxy, the only published port, which forwards `/preview/<doc>/<nonce>.<hmac>/` to the right container after checking the token; the clone arrives as a git bundle the worker fetched and wrote in by `docker exec`

The canvas connects to the container's IP directly

Measured: an internal-only container gets no published port. A proxy route to Gitea was rejected: customer code would reach every repo the token reads.

Next: Lesson 6, after epic 6. Durability: the op journal and snapshots, so a sync node killed mid-edit loses no acknowledged op and duplicates none; and the id an engineer brings back is refused from the journal, not from git log.